# CAVE

## The Complete System Guide

Compressed Atomic Verb Expressions

Repository version 0.39.2

# About this book

CAVE is a small plain-text language for writing down what you know, one claim per line, and a command-line tool that stores those claims and lets you ask questions across them. This book teaches both, from the first three-word claim to a store that reads its own inputs, concludes, acts, and explains itself.

The book is organized like a course. Part I is the language: how to write a claim and everything a claim can carry. Part II is the store: what happens to a claim once it is recorded, how belief changes, and how to ask. Part III is about getting knowledge in without typing it. Part IV is about concluding and acting. Part V covers sharing what a store knows, with people, agents, and other stores. Part VI looks under the hood. Each chapter introduces a few ideas, shows them on one running example, and closes with a short summary.

**The running example.** Most chapters follow a small coffee roastery: the cooperatives it buys from, the lots of green coffee, the blends it roasts, and the cafes it sells to. It is deliberately ordinary. Nothing in CAVE is specific to software, and a domain everyone can picture keeps the attention on the ideas.

**Every session in this book is real.** A block that starts with a `$` prompt was run against the `cave` command that ships with this repository version, and its output is what the tool printed; when a command exits with a nonzero status, the session shows it as a final `[exit N]` line, and a pipeline counts as failed when its `cave` stage fails (the sessions run with `pipefail` set). A test replays every such session and fails when the recorded output or exit status drifts; the few sessions that need a language model, a browser, a long-running server, or the optional Z3 solver package are marked in the source and are not replayed. Files shown with a filename caption are the exact files the sessions use; `book/fixtures/` in the repository holds the shared ones, so you can copy that directory and follow along.

**Where the rules live.** The normative specification is split across four skill files in the repository's `.claude/skills/` directory, with numbered sections (§). This book explains and motivates; when it names a section, that section has the last word on exact wording.

Build target: Typst 0.15.0. Project version: 0.39.2.

# Contents

**The language** ..... **7**
1 A First Claim ..... 8
1.1 The tool ..... 8
1.2 What the book covers ..... 9
2 Claims ..... 10
2.1 The roastery ..... 10
2.2 Two shapes of payload ..... 11
2.3 Names ..... 11
2.4 Literals ..... 11
2.5 Comments ..... 12
2.6 Lint before you load ..... 12
3 Verbs ..... 13
3.1 Two verbs are enough to start ..... 13
3.2 The standard verbs ..... 13
3.3 Declaring your own ..... 13
3.4 Reverse readings ..... 14
3.5 Negation ..... 14
3.6 Renaming a verb ..... 15
4 Context, Tags, and Confidence ..... 16
4.1 Context: `@name` ..... 16
4.2 Tags: `#tag` and `#key:value` ..... 16
4.3 The two-lane rule ..... 17
4.4 Confidence: `@ N%` ..... 17
4.5 Importance: `!` ..... 18
4.6 Comments, once more ..... 18
5 Values, Units, and Uncertainty ..... 19
5.1 Typed values ..... 19
5.2 Comparing values in queries ..... 19
5.3 Approximate values: `~` ..... 19
5.4 Spread: `+/- delta` ..... 19
5.5 Two kinds of uncertainty ..... 20
5.6 Competing hypotheses ..... 20
5.7 What a trajectory is ..... 20
6 Indentation ..... 21
6.1 Qualifiers ..... 21
6.2 Continuation ..... 21
6.3 Grouped claims ..... 22
6.4 Explicit full claims ..... 22
6.5 Prefix headers ..... 22
6.6 What canonical output looks like ..... 23
**The store** ..... **24**
7 Belief That Only Grows ..... 25
7.1 Append, never update ..... 25
7.2 Claim keys and current belief ..... 25
7.3 Retraction ..... 25
7.4 Contradictions are allowed ..... 26
7.5 The history, as text ..... 26
7.6 Looking back: `--as-of` ..... 26
8 Provenance ..... 28

8.1 Every append is stamped ... 28
8.2 Four dimensions behind one syntax ... 29
8.3 Citing the exact line ... 29
8.4 Audience labels ... 29
8.5 What permanence means ... 30
8.6 Exact backups ... 30
9 Asking Questions ... 31
9.1 Patterns ... 31
9.2 Filters ... 32
9.3 Reading backwards ... 32
9.4 Following a chain ... 32
9.5 Joining patterns ... 33
9.6 Current, all, and the rest ... 34
9.7 Paging and JSON ... 34
9.8 Words, not patterns ... 34
9.9 A file is a store ... 35
9.10 SQL, when you need it ... 35
10 Time in the World ... 36
10.1 Time contexts ... 36
10.2 Anchoring a query: `--at` ... 36
10.3 Trajectories: `A -> B` ... 37
10.4 Both axes at once ... 37
11 When Sources Disagree ... 39
11.1 A contest ... 39
11.2 What competes with what ... 39
11.3 The policy ... 39
11.4 Declaring policy in-band ... 40
11.5 Where resolution applies ... 40
12 One Entity, Many Names ... 41
12.1 Declaring an alias ... 41
12.2 The closure ... 41
12.3 Finding candidates ... 42
13 Shape and Health ... 43
13.1 Expectations are claims ... 43
13.2 The health report ... 43
13.3 The write gate ... 44
13.4 A typed client ... 44
**Getting knowledge in ... 46**
14 Structured Data Without a Model ... 47
14.1 A template is a CAVE file with holes ... 47
14.2 Records remember themselves ... 48
14.3 When the source changes ... 48
14.4 Continuous and query-time reads ... 49
14.5 A one-line mapping, and SQL first ... 49
14.6 Declaring a source ... 49
15 Letting a Model Read ... 51
15.1 The agent contract ... 51
15.2 What the prompt asks for ... 51
15.3 Running the pipeline without a model ... 52
15.4 Operating modes for a model ... 52
16 Measuring an Extraction ... 54

16.1 A fixture ..... 54
16.2 Scoring ..... 54
16.3 Making it a gate ..... 55
16.4 Scoring a reconstruction ..... 55
**Concluding and acting ..... 57**
17 Rules ..... 58
17.1 The rule line ..... 58
17.2 Firing ..... 58
17.3 Rules are claims, and conclusions carry their reasons ..... 59
17.4 Re-running is safe ..... 59
18 Actions ..... 61
18.1 Declaring an action ..... 61
18.2 Executing ..... 61
18.3 What an execution leaves behind ..... 62
18.4 The gate, and hooks ..... 63
18.5 Serving actions to agents ..... 63
19 Automations ..... 64
19.1 Declaring an automation ..... 64
19.2 Arming, and the first cycle ..... 64
19.3 Something happens ..... 65
19.4 Why the loop cannot run away ..... 65
19.5 Running it ..... 66
20 Reconstruction ..... 67
20.1 A symptom ..... 67
20.2 The policy ..... 67
20.3 Why not a vector search ..... 68
**Sharing what the store knows ..... 69**
21 Documents That Cite Their Claims ..... 70
21.1 A template ..... 70
21.2 Rendering ..... 70
21.3 Splices are deterministic or nothing ..... 71
21.4 The same report, another day ..... 72
22 Looking at the Store ..... 73
22.1 Starting it ..... 73
22.2 The views ..... 73
22.3 The promises ..... 74
22.4 Scripting it ..... 74
23 Agents Over MCP ..... 75
23.1 The server ..... 75
23.2 The tools ..... 75
23.3 Provenance for agents ..... 76
23.4 Scoping what an agent may do ..... 76
23.5 Actions as the write vocabulary ..... 77
23.6 Driving an agent from the command line ..... 77
24 Two Stores Become One ..... 78
24.1 Rows have global identity ..... 78
24.2 The merge is a claim ..... 78
24.3 The receive rule ..... 79
24.4 Text that carries identity ..... 79
24.5 The store under git ..... 79
**Under the hood ..... 81**

25 How Claims Are Stored ..... 82
25.1 The tables ..... 82
25.2 From a line to rows ..... 82
25.3 Inverses are views ..... 83
25.4 Current belief in SQL ..... 83
25.5 Schema versions ..... 83
26 How the Pieces Fit ..... 85
26.1 Three boundaries ..... 85
26.2 The layers ..... 85
26.3 One process boundary ..... 86
26.4 One grammar, four renderers ..... 86
26.5 The invariants ..... 86
26.6 Boundaries that will stay ..... 87
27 Formal Reasoning ..... 88
27.1 Scenario inputs ..... 88
27.2 Solver-neutral models ..... 88
27.3 The Z3 adapter ..... 88
27.4 Explanations and recording ..... 89
28 Running a Store for Real ..... 90
28.1 Habits ..... 90
28.2 Security boundaries ..... 90
28.3 Performance ..... 90
28.4 Checking an installation ..... 91
28.5 Signals, exit codes, and errors ..... 91
**Appendices ..... 92**
29 Command Reference ..... 93
30 Syntax Card ..... 95
31 Where the Rules Live ..... 97
31.1 Chapter to section ..... 97

PART

# The language

How to write a claim, and everything a claim can carry: names, verbs, attributes, values with units, confidence, context, tags, and the indentation that groups related lines.

CHAPTER 1

# A First Claim

Most of what a team knows never gets written down in a form a program can use. It lives in chat threads, in the heads of two people, in a wiki page that was right in March. When it does get written down, it is prose: easy to write, impossible to query, and silently wrong the moment the world moves on.

CAVE starts from a small observation. Almost every useful fact can be said in three words: a thing, a relationship, another thing.

```
kaffa-coop SUPPLIES lot/yirgacheffe-26
```

That line is a **claim**. The thing on the left is the **subject**, the uppercase word is the **verb**, and the thing on the right is the **object**. It is short enough to type in a chat message, regular enough for a program to index, and plain enough that a language model can produce thousands of them from a pile of documents without being told much.

Everything else in CAVE is built by adding small, optional pieces to this shape. A claim can name where it came from and how sure you are:

```
lot/yirgacheffe-26 HAS score: 87 @src:cupping/june @ 90%
```

It can carry a value with a unit, a tag, or a comment. It can be qualified by another claim indented beneath it. But the three-word core never changes, and that is what makes the rest of the system possible: a store of claims is a graph you can walk, a history you can replay, and a set of patterns you can match.

## 1.1 The tool

The `cave` command reads claims, keeps them in a local store, and answers questions about them. Install it once:

```
pnpm i -g @cavelang/cli
```

Then try the smallest possible loop. `cave parse` checks text without storing anything:

```
$ echo 'kaffa-coop SUPPLIES lot/yirgacheffe-26' | cave parse
ok: 1 claim, 1 blank
```

`cave add` records claims in a store, creating the database file if it does not exist. `cave query` asks a question in the same three-word shape, with a `?variable` where the answer should go:

```
$ printf '%s\n' 'kaffa-coop SUPPLIES lot/yirgacheffe-26' \
    'la-cima SUPPLIES lot/huila-26' | cave add --db first.db
added 2 claim(s), 0 edge(s)

$ cave query --db first.db '?who SUPPLIES lot/huila-26'
?who = la-cima
```

Three things just happened that will matter for the rest of the book. The verb `SUPPLIES` was accepted although nobody defined it; CAVE lets you use a vocabulary before you formalize it. The claims were **appended**, not inserted into a table with a primary key; a store only ever grows, so it remembers how a belief changed. And the question had the same shape as the answer; there is no separate query language to learn beyond a `?` in front of a name.

## 1.2 What the book covers

The chapters that follow build up the language one piece at a time, then move into the store and the tools around it. If you only want to write claims, Part I is enough. If you want to keep a store, Part II. Parts III to V are about letting programs and models do the reading, concluding, and reporting for you. Part VI is for the curious and the operators.

Every session with a `$` prompt in this book was run against the real tool, and a test keeps it that way for all but the few that need a language model, a browser, a long-running server, or the optional Z3 solver package. When the output changes, the book changes with it.

> **In short.** A claim is `subject VERB object`. `cave parse` lints, `cave add` appends to a store, and `cave query` asks with `?variables`. Nothing is ever edited in place.

CHAPTER 2

# Claims

This chapter introduces the roastery that the rest of the book follows, and uses it to explain exactly what a claim line may contain: names, the two payload shapes, literals, and comments.

## 2.1 The roastery

A small roastery buys green coffee from cooperatives, roasts it into blends and single origins, and sells those to a few cafes. Here is that world in CAVE, the file every later chapter starts from:

roastery.cave

```
; the roastery: what we buy, what we roast, who we sell to

SUPPLIES IS verb ; supplier X sells us green coffee lot Y
SUPPLIES REVERSE SUPPLIED-BY
STOCKS IS verb ; cafe X keeps coffee Y on its menu
STOCKS REVERSE STOCKED-BY

kaffa-coop IS supplier
la-cima IS supplier
kaffa-coop HAS country: ethiopia
la-cima HAS country: colombia

lot/yirgacheffe-26 IS lot
lot/huila-26 IS lot
lot/santa-ana-26 IS lot
kaffa-coop SUPPLIES lot/yirgacheffe-26
la-cima SUPPLIES lot/huila-26
la-cima SUPPLIES lot/santa-ana-26

lot/yirgacheffe-26 HAS
  process: washed
  price: 9.40 USD/kg
  score: 87 @src:cupping/june
lot/huila-26 HAS
  process: natural
  price: 7.80 USD/kg
  score: 84 @src:cupping/june
lot/santa-ana-26 HAS
  process: washed
  price: 8.10 USD/kg
  score: 85 @src:cupping/june ; clean, but nothing remarkable

coffee/morning-blend IS coffee
coffee/morning-blend USES lot/huila-26
coffee/morning-blend USES lot/santa-ana-26
coffee/yirgacheffe IS coffee
coffee/yirgacheffe USES lot/yirgacheffe-26

cafe/north IS cafe
cafe/harbour IS cafe
cafe/north STOCKS coffee/morning-blend
cafe/north STOCKS coffee/yirgacheffe
cafe/harbour STOCKS coffee/morning-blend
```

Read it top to bottom and it tells a story: two suppliers, three lots with a price and a cupping score each, two coffees made from those lots, and two cafes that stock them. The first four lines declare two verbs of our own and their reverse readings; Chapter 3 explains them. The indented blocks under `HAS` are a shorthand for repeating the prefix; Chapter 6 explains that. For now, lint the file and load it:

```
$ cave parse roastery.cave
ok: 1 comment, 7 blank, 33 claim, 3 prefix

$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)
```

## 2.2 Two shapes of payload

A claim is a subject, a verb, and a **payload**. The payload comes in two shapes, and the difference is one colon.

A **relation** connects two things:

```
la-cima SUPPLIES lot/huila-26
cafe/north STOCKS coffee/yirgacheffe
```

An **attribute** gives a thing a named property with a value:

```
la-cima HAS country: colombia
lot/huila-26 HAS price: 7.80 USD/kg
```

The colon is not decoration. Without it, `lot/huila-26 HAS price 7.80 USD/kg` could mean an object called `price 7.80 USD/kg` or an attribute `price` with a value; a query engine has to know which. So attribute claims always use `attribute: value`, and the parser only reads a colon that way in payload position. There is a third, minor shape for a bare measurement, the **metric** claim, which is `IS` with a value and no attribute name:

```
roaster/temperature IS 205 C
```

Most of the time you will want `HAS attribute: value` instead, because it names what is being measured.

## 2.3 Names

A name is a compact, lowercase, kebab-case token. A slash scopes it, and up to three segments read naturally as `domain/entity/aspect`:

```
lot/huila-26
cafe/north
roaster/drum/temperature
```

Proper nouns keep their capitals (`PostgreSQL`, `Colombia`), but the roastery file writes `colombia` because it is a value, not an entity anyone will query by name. The single most valuable habit in CAVE is spelling the same thing the same way every time. `la-cima` and `La Cima` and `lacima` are three entities to a store until you tell it otherwise (Chapter 12 shows how), and no amount of metadata makes up for a name that drifts.

Two characters do double duty and are worth a moment. A slash inside a name means scope; a slash after a number means "per", as in `USD/kg`. A colon in a payload separates attribute from value; a colon inside a `#tag` splits key from value (Chapter 4). In both cases the position decides, and the disambiguation is one character of lookahead.

## 2.4 Literals

Sometimes the object is not a name but a piece of exact text. Backticks keep code-like text exactly as written, and double quotes hold natural language:

```
grinder/harbour LOGS `E_BURR_TEMP`
step/1 IS "rest the beans for three days"
```

Inside either kind of literal nothing is special: a `;` or `@` or `#` is just a character. Use literals when a name would be a lie, and names everywhere else, because names are what queries join on.

## 2.5 Comments

A semicolon starts a comment, and the comment is **part of the claim**: it is stored with the row, exported with it, and searchable.

```
lot/santa-ana-26 HAS score: 85 @src:cupping/june ; clean, but nothing remarkable
```

A comment can be longer than one line. A block of `;` lines directly above a claim is part of that claim's comment, and the trailing `; note` on the claim line, if there is one, is its last line. Each line keeps everything after the semicolon and one space, so a comment can hold indented text such as a code sample or a list without losing its shape:

`cupping-notes.cave`

```
; the June cupping, scored by two of us and averaged

; the santa-ana lot cupped clean, but the finish
; was flat by the third cup
lot/santa-ana-26 HAS score: 85 @src:cupping/june ; averaged from two scores
```

The first line is separated from the claim by a blank line, so it stays a note about the file. The block under it belongs to the claim, and it comes back with the claim wherever the claim is printed:

```
$ cave add --db notes.db cupping-notes.cave
added 1 claim(s), 0 edge(s)

$ cave query --db notes.db 'lot/santa-ana-26 HAS score: ?s'
; the santa-ana lot cupped clean, but the finish
; was flat by the third cup
?s = 85  ; averaged from two scores
```

A comment with nothing but a blank line between it and the next claim is documentation for the file and is not stored, which is what the first line of `roastery.cave` is. Comments are the escape hatch for nuance that does not fit a triple, and the file above uses them sparingly on purpose. When you catch yourself writing a paragraph after a semicolon, the paragraph probably contains three more claims.

## 2.6 Lint before you load

`cave parse` reads text and reports what it found: how many claims, blank lines, comments, and prefix headers. When a line is malformed it prints the problem with its line number and exits with status 1, which makes it a cheap pre-commit check for hand-written files.

```
$ printf 'la-cima SUPPLIES\n' | cave parse
1 invalid, 1 blank
line 1: missing object after SUPPLIES (the minimum line is "entity VERB object", spec §2.2)
[exit 1]
```

`cave add` is lenient by default: valid lines land, problems go to standard error. `cave add --strict` refuses the whole file if anything is wrong, which is the right setting for files that other people review.

> **In short.** A claim is `subject VERB object` or `subject HAS attribute: value`. Concept names are lowercase; proper names keep their casing. Use `/` for scope and spell each entity consistently. Backticks and quotes hold exact text; `;` starts a comment that is stored with the claim. `cave parse` lints, `cave add` loads.

CHAPTER 3

# Verbs

The verb is the only part of a claim that carries fixed meaning. This chapter covers the two verbs the language is built on, the standard set worth knowing by heart, how to declare your own, and the two declarations that make a vocabulary evolve gracefully: `REVERSE` and `RENAMED-TO`.

## 3.1 Two verbs are enough to start

CAVE bootstraps from `IS` and `HAS`. `IS` gives a thing a type, a state, or a bare measurement; `HAS` gives it a property.

```
la-cima IS supplier
la-cima HAS country: colombia
roaster/drum IS warming-up
```

Everything else, including the standard verbs and the declarations below, is defined on top of these two. You could write a useful store with nothing else. You would just be giving up graph quality: `coffee/morning-blend USES lot/huila-26` tells a query engine something that `coffee/morning-blend HAS ingredient: lot/huila-26` does not, because `USES` is a relation between two entities that can be walked in either direction.

## 3.2 The standard verbs

The standard set is small and grouped by what it describes. Use one of these when it fits; the graph gets better every time two claims share a verb.

| Family | Verbs | Example |
|---|---|---|
| Identity and taxonomy | `IS`, `EXTENDS`, `ALIAS`, `LIKE`, `EXISTS` | `natural EXTENDS process` |
| Causation and change | `CAUSE`, `FIX`, `BECOMES` | `stale-lot CAUSE flat-taste` |
| Dependency and production | `NEEDS`, `USES`, `YIELDS`, `ENABLES`, `BLOCKS` | `roast/light YIELDS coffee/yirgacheffe` |
| Structure and ordering | `CONTAINS`, `PRECEDES`, `EXCEEDS`, `VS` | `resting PRECEDES packing` |
| Qualifiers | `WHEN`, `UNLESS`, `VIA`, `BECAUSE` | indented under another claim (Chapter 6) |

Two conventions keep causal claims readable. Causation runs cause to effect: `stale-lot CAUSE flat-taste`, never `flat-taste CAUSE stale-lot`. And `ALIAS` means the same entity under another name, while `LIKE` means similar but distinct; the store treats them very differently (Chapter 12).

## 3.3 Declaring your own

When no standard verb fits, declare one. The declaration is itself a claim on the bootstrap verb, so it lives in the same file as the facts that use it, and it is stored, exported, and attributed like any other claim:

```
SUPPLIES IS verb ; supplier X sells us green coffee lot Y
```

The comment is the verb's documentation and the convention for it is worth copying: name the two sides as X and Y. You can add more claims about the verb (`SUPPLIES HAS domain: purchasing`, `SUPPLIES LIKE YIELDS`) if you want to, but the one line above is all the engine needs. Keep extensions rare. If an extraction from one document invents more than three new verbs, the document is probably being modelled too specifically.

A verb you did not declare still works: Chapter 1 used `SUPPLIES` before declaring it. Declaring it buys you documentation, a place to hang the reverse name, and a reviewable moment when the vocabulary changed.

## 3.4 Reverse readings

Every relation can be read from either end. The roastery file declares that reading for both of its verbs:

```
SUPPLIES REVERSE SUPPLIED-BY
STOCKS REVERSE STOCKED-BY
```

Now both spellings query, and the answers come from the same stored rows:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ cave query --db roastery.db 'la-cima SUPPLIES ?lot'
?lot = lot/huila-26
?lot = lot/santa-ana-26

$ cave query --db roastery.db 'lot/huila-26 SUPPLIED-BY ?who'
?who = la-cima
```

The standard verbs come with their inverses declared: `USES` reads back as `USED-BY`, `CONTAINS` as `PART-OF`, `CAUSE` as `CAUSED-BY`, `NEEDS` as `NEEDED-BY`, `PRECEDES` as `FOLLOWS`, and so on. So a question the file never states directly is still one line:

```
$ cave query --db roastery.db 'lot/huila-26 USED-BY ?coffee'
?coffee = coffee/morning-blend
```

What matters is what `REVERSE` does **not** do: it does not create a second row. The left verb of the declaration is the **primary** direction, and a line written with the inverse is normalized to primary form before it is stored or keyed. Write the same fact backwards and you are writing to the same fact:

```
$ echo 'lot/huila-26 SUPPLIED-BY la-cima @ 90% ; contract not yet signed' | cave add --db
roastery.db
added 1 claim(s), 0 edge(s)

$ cave query --db roastery.db 'la-cima SUPPLIES lot/huila-26' --all
la-cima SUPPLIES lot/huila-26
lot/huila-26 SUPPLIED-BY la-cima @ 90% ; contract not yet signed
```

Two rows, one belief series, each shown as it was written: the second row is now the current belief about one fact with two names. Confidence, negation, and history all belong to the fact, not to the direction you happened to read it in.

> **Why not store both directions?**
> Materializing inverses would double every row, split each fact's belief history in two, and double the work of resolving contradictions. A reverse reading is a query-time view over an index the store already has. See spec §5.5 and §13.3.

## 3.5 Negation

`NOT` right after the verb negates any relation. It records that something is **not** the case, which is a different and stronger statement than having no claim at all:

```
cafe/harbour STOCKS NOT coffee/yirgacheffe ; they asked, we said no
roaster/drum IS NOT contaminated @ 95%
```

Negation is stored on the same row as the positive form would be, with a flag, and it reads back through the inverse just as naturally (`coffee/yirgacheffe STOCKED-BY NOT cafe/harbour`). Chapter 7 contrasts negation

with **retraction**, which is a positive claim at zero confidence: one says "this is false", the other says "we no longer assert this".

## 3.6 Renaming a verb

Vocabularies drift. Suppose the roastery decides `STOCKS` should have been `CARRIES`. A rename is a directional declaration:

```
$ echo 'STOCKS RENAMED-TO CARRIES' | cave add --db roastery.db
added 1 claim(s), 0 edge(s)

$ cave query --db roastery.db 'cafe/north CARRIES ?coffee'
?coffee = coffee/morning-blend
?coffee = coffee/yirgacheffe
```

The old spelling remains accepted, the new one is preferred, and both resolve to the **original** verb as the stable storage identity. Claims written before the rename and after it share one belief history, and nothing in the store is rewritten. Renames chain linearly (`A RENAMED-TO B`, then `B RENAMED-TO C`), but they cannot branch, cycle, or collide with a verb that already exists; the first valid declaration wins. Because the declaration is a claim with a transaction time, a query asked as of a date before the rename does not know the new name (Chapter 7 introduces as-of reads).

> **In short.** `IS` and `HAS` bootstrap everything. Prefer the standard verbs; declare your own with `X IS verb ; X does what to Y`. `A REVERSE B` makes one stored fact readable and writable under two names. `NOT` asserts a negative. `OLD RENAMED-TO NEW` evolves a name without rewriting history.

CHAPTER 4

# Context, Tags, and Confidence

A three-word claim says **what**. The metadata after it says **where**, **from whom**, **how surely**, and **filed under what**. This chapter introduces the four qualifiers that most claims end up carrying, and the one rule that decides which of two similar mechanisms to use.

Here is the full anatomy of a line, with every optional piece present:

```
subject VERB object +/- delta @context #tag @ 90% ! ; comment
|       |    |      |         |        |    |     |  |
|       |    |      |         |        |    |     |  └ persisted prose
|       |    |      |         |        |    |     └ importance marker
|       |    |      |         |        |    └ claim confidence
|       |    |      |         |        └ tag: flat #tag or scoped #key:value
|       |    |      |         └ scope, location, time, or source
|       |    |      └ value uncertainty (Chapter 5)
|       |    └ the object, or attribute: value
|       └ relationship (UPPERCASE)
└ the subject
```

All suffixes are optional. Contexts and tags may repeat.

## 4.1 Context: `@name`

A context scopes a claim. It is an `@` immediately followed by a name, with no space:

```
lot/huila-26 HAS score: 84 @src:cupping/june
coffee/morning-blend HAS defect: flat @cafe/harbour
roaster/drum HAS temperature: 205 C @time:2026-08-14T07:30Z
```

Four prefixes are recommended because tools read them: `@src:` names the source a claim came from, `@time:` an event time, `@loc:` a location, and `@scope:` a logical partition. Bare contexts like `@production` or `@cafe/harbour` are fine too. A date-like context, bare or after `@time:`, is a **time context** that valid-time queries understand (Chapter 10).

Context is part of a claim's identity. `lot/huila-26 HAS score: 84 @src:cupping/june` and the same score `@src:cupping/july` are two facts with two histories, not one fact stated twice. That is exactly what you want for sources: two graders can disagree, and the store keeps both (Chapter 11 shows how to pick a winner when you need one).

## 4.2 Tags: `#tag` and `#key:value`

A tag classifies the claim. It is flat (`#organic`) or scoped (`#certification:organic`); a flat tag is a scoped tag with no value.

```
lot/yirgacheffe-26 HAS certification: organic #verified @src:kaffa-coop/docs
lot/santa-ana-26 HAS defect: quakers #severity:low @src:cupping/june
```

Tags filter queries directly. Load the store and ask for verified certifications, then for every low-severity defect:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ printf '%s\n' \
```

```
    'lot/yirgacheffe-26 HAS certification: organic #verified @src:kaffa-coop/docs' \
    'lot/huila-26 HAS certification: organic @src:la-cima/email' \
    'lot/santa-ana-26 HAS defect: quakers #severity:low @src:cupping/june' \
    | cave add --db roastery.db
added 3 claim(s), 0 edge(s)

$ cave query --db roastery.db '?lot HAS certification: organic #verified'
?lot = lot/yirgacheffe-26

$ cave query --db roastery.db '?lot HAS defect: ?d #severity:low'
?lot = lot/santa-ana-26  ?d = quakers
```

Tags are stored beside the claim, not in its identity, and they carry no history of their own. Which brings us to the rule.

## 4.3 The two-lane rule

CAVE has two ways to classify, and they answer different questions. An **entity facet** is an attribute claim: `lot/huila-26 HAS topic: colombia`. It classifies the entity, and because it is a claim it has its own belief history and can be retracted later. A **claim facet** is a tag: `... #topic:colombia`. It files the claim, and it has no life of its own.

The writer's test is: **does this classification deserve its own history?** A lot's certification can be granted and later withdrawn, so it is an attribute. A note that this particular score belongs to the June cupping is about the claim, so it is a tag or a context. Collapsing the two would force spurious history onto tags or strip entity memberships of theirs.

Topics follow from the rule without new syntax. A topic is an ordinary entity, conventionally under `topic/`, and membership is `CONTAINS`:

```
topic/ethiopia IS topic
topic/ethiopia CONTAINS kaffa-coop
topic/ethiopia CONTAINS lot/yirgacheffe-26
```

Since `CONTAINS` has the inverse `PART-OF`, "which topics is this lot in" is a query with no extra rows.

## 4.4 Confidence: `@ N%`

A space after the `@` and a trailing `%` make a confidence, and the space is the whole difference between it and a context:

```
coffee/morning-blend HAS defect: flat @cafe/harbour @ 70% ; two complaints this week
```

Omitted confidence means 100 percent: directly observed, certain for practical purposes. The scale is meant to be used coarsely:

| Confidence | Reading |
|---|---|
| `@ 100%` | Directly observed; the default, so omit it. |
| `@ 90%` | High confidence, reliable source. |
| `@ 70%` | Likely; several signals agree. |
| `@ 50%` | Could go either way. |
| `@ 30%` | Unlikely but plausible. |
| `@ 0%` | Rejected. Also how a claim is retracted (Chapter 7). |

Canonical exports preserve the full stored confidence, even when that requires a long decimal percentage. For example, `@ 50.0001%` remains `@ 50.0001%` after export and import. Display rounding must not change the stored belief.

Confidence filters queries with a `WHERE` line, passed as a second argument:

```
$ printf '%s\n' \
    'coffee/morning-blend HAS defect: flat @cafe/harbour @ 70% ; two complaints this week' \
    'coffee/morning-blend HAS defect: sour @cafe/north @ 20% ; one vague remark' \
    | cave add --db roastery.db
added 2 claim(s), 0 edge(s)

$ cave query --db roastery.db 'coffee/morning-blend HAS defect: ?d'
?d = flat  ; two complaints this week
?d = sour  ; one vague remark

$ cave query --db roastery.db 'coffee/morning-blend HAS defect: ?d' 'WHERE conf >= 0.5'
?d = flat  ; two complaints this week
```

A context in the pattern narrows it the same way a tag does: `coffee/morning-blend HAS defect: ?d @cafe/north` matches only the second claim.

Confidence belongs to the **claim**, not to the value. A measurement can be imprecise while the claim about it is certain, and the other way round; Chapter 5 keeps the two apart.

## 4.5 Importance: `!`

A bare `!` marks a claim as important. It is a flag, nothing more, but it is stored and indexed, and a store with a few hundred claims benefits from having ten of them flagged:

```
roaster/drum HAS service-due: 2026-09-01 !
```

## 4.6 Comments, once more

The comment after `;` rides with the claim through storage, export, search, and reports, and so does a block of `;` lines directly above the claim (Chapter 2). Everything before this chapter used it for the small rationale a triple cannot carry, and that is the right amount: a comment is where you write **why**, not where you write the next three claims.

> **In short.** `@name` scopes a claim and is part of its identity; `@src:`, `@time:`, `@loc:`, and `@scope:` are the recommended prefixes. `#tag` and `#key:value` file the claim and carry no history. `@ N%` with a space is confidence, default 100. `!` flags importance. Use an attribute when a classification deserves its own history and a tag when it does not.

CHAPTER 5

# Values, Units, and Uncertainty

Attribute values are where CAVE meets numbers. This chapter explains how a value is written, how units and multipliers are read, how to say that a measurement is approximate or has a spread, and why that spread is a different thing from confidence.

## 5.1 Typed values

A value is a number, a number with a unit, a date-like token, a name, or a literal. The parser stores the original text and, when it can, a parsed number and a normalized unit beside it:

```
lot/huila-26 HAS price: 7.80 USD/kg
lot/huila-26 HAS moisture: 10.8%
kaffa-coop HAS capacity: 120K kg/yr
roaster/drum HAS warm-up: 25min
lot/huila-26 HAS harvest: 2026-Q1
```

The rules are few. A simple unit glues to its number (`25min`, `205C`, `3600s`); a compound unit takes a space (`7.80 USD/kg`, `120K kg/yr`); a slash inside a unit means "per"; `%` is a unit; and the multipliers `K`, `M`, `B`, and `T` scale the number so that `120K` is stored as 120000 and `2.5B` as 2500000000. Units the engine has never seen pass through verbatim, so `205 C` and `18 bags` work without registration.

## 5.2 Comparing values in queries

Because numbers and units are parsed at write time, a query can compare them. `WHERE value` takes an operator and a value, unit included:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ cave query --db roastery.db '?lot HAS price: ?p' 'WHERE value < 9 USD/kg'
?lot = lot/huila-26  ?p = 7.80 USD/kg
?lot = lot/santa-ana-26  ?p = 8.10 USD/kg

$ cave query --db roastery.db '?lot HAS score: ?s' 'WHERE value >= 85'
?lot = lot/yirgacheffe-26  ?s = 87
?lot = lot/santa-ana-26  ?s = 85  ; clean, but nothing remarkable
```

Comparisons require compatible units: a filter in `USD/kg` never matches a value in `EUR/kg`, and no conversion is attempted. Conversions are policy, and CAVE keeps policy out of the storage layer.

## 5.3 Approximate values: `~`

A tilde before a value marks it as approximate. The number is still parsed and still comparable; the flag records that nobody measured it precisely:

```
kaffa-coop HAS capacity: ~120K kg/yr
cafe/north HAS weekly-volume: ~40 kg/wk
```

## 5.4 Spread: `+/- delta`

When a measurement has a known spread, write it with `+/-` and the same unit. The default reading is two standard deviations, which matches how people say "ten point eight, plus or minus point three":

```
lot/huila-26 HAS moisture: 10.8% +/- 0.3%
lot/yirgacheffe-26 HAS density: 0.72 g/ml +/- 0.02 g/ml (1σ)
```

The optional `(Nσ)` says which level the delta is quoted at. If the interval is Δ at kσ, then σ is Δ/k; the store keeps the delta text, the parsed delta, and the sigma level as separate columns so a later computation can use them.

## 5.5 Two kinds of uncertainty

This is the idea worth slowing down for. A value's spread is **aleatory** uncertainty: the quantity itself is imprecise. A claim's confidence is **epistemic** uncertainty: how much you believe the assertion. They are independent, and CAVE writes them in different places so they never get confused:

```
; sure of the measurement, which is itself imprecise

lot/huila-26 HAS moisture: ~10.8% +/- 0.3% @src:meter @ 95%

; a precise number from a source we do not trust much

lot/huila-26 HAS moisture: 11.4% @src:la-cima/email @ 30%
```

The first line says: we measured it ourselves, the meter is not very precise, and we are nearly certain of the reading. The second says: someone told us a suspiciously exact number. A reader, human or program, can weigh those two lines properly only because the language kept the two uncertainties apart.

**Fusing estimates**
Given several estimates of one quantity, each with a spread and a confidence, a precision-weighted average gives a fused value and spread; confidence acts as a weight. The MCP server offers this as the `cave_fuse` tool so that agents do not do the arithmetic in tokens. The math is spec §10.1; it is an implementation layer, not syntax. The library validates finite estimates and confidence in `[0, 1]`, scales the arithmetic to avoid intermediate overflow, and reports an error when the posterior cannot be represented with finite positive precision and spread.

## 5.6 Competing hypotheses

Numbers are not the only place uncertainty lives. When several explanations compete, give each its own entity rather than overwriting one claim, so that new evidence can update each hypothesis separately:

```
stale-lot CAUSE flat-taste @ 50%
grinder/harbour CAUSE flat-taste @ 30%
water/harbour CAUSE flat-taste @ 20%
```

The confidences of an exhaustive set should sum to about 100 percent. When the grinder is serviced and the taste does not change, append a new assessment for each line; the old ones remain in the history, which is the subject of Chapter 7.

## 5.7 What a trajectory is

One more value form exists, and it belongs to time: `A -> B` names a value that moves from one number to another across a time range, such as `price:`
`7.80 -> 8.60 USD/kg @2026..2027`. Chapter 10 introduces it together with the query that reads it at an instant.

> **In short.** Values carry parsed numbers and normalized units, so `WHERE value` compares them, but only within one unit. ~ marks an approximate value, `+/- delta (Nσ)` a spread. Spread is about the quantity; `@ N%` is about the claim. Keep competing hypotheses as separate entities.

CHAPTER 6

# Indentation

One claim per line is the rule, and it stays the rule. Indentation does not break it. An indented line is still a claim; what indentation adds is a relationship between that claim and the less-indented line above it, its **parent**. There are three kinds of indented line, told apart by what the line starts with, plus one shorthand that is not a claim at all.

## 6.1 Qualifiers

An indented line that starts with `WHEN`, `UNLESS`, `VIA`, or `BECAUSE` is a **qualifier**. It states a condition, a mechanism, or the evidence for its parent, and the store records an **edge** from the parent claim to the qualifier claim:

`tasting.cave`

```
; the July cupping: what we tasted, and what we think explains it

coffee/morning-blend HAS defect: flat @cafe/harbour @ 70%
  WHEN grinder/harbour HAS burr-age: 14mo
  BECAUSE cupping/july

stale-lot CAUSE flat-taste @ 50%
  UNLESS lot/huila-26 HAS roast-date: 2026-08-10
```

Each qualifier is stored as a claim in its own right, with its own key and history, and the edge is stored beside it with the role (`WHEN`, `BECAUSE`) that the verb named. `UNLESS x` is accepted as a spelling of `WHEN NOT x`; canonical output prefers `WHEN NOT`. A qualifier never inherits anything from its parent and is not affected by reverse declarations: it is exactly the claim it says it is.

```
$ cave add --db roastery.db roastery.cave tasting.cave
added 38 claim(s), 3 edge(s)
```

Thirty-eight claims because a qualifier is a claim; three edges because that is how many qualifiers there were. The edges are what make derivation (Chapter 17) and reports (Chapter 21) able to answer **why is this believed**.

## 6.2 Continuation

An indented line that starts with a bare verb, one of the ordinary relation verbs rather than a qualifier verb, is a **continuation**: it inherits the parent's subject and becomes an independent sibling claim.

```
kaffa-coop SUPPLIES lot/yirgacheffe-26
  SUPPLIES lot/guji-26
  SUPPLIES lot/sidama-26
```

That is three claims with the subject `kaffa-coop`. The inherited endpoint follows the verb's direction: when the continuation uses an inverse verb, the parent's subject lands in **object** position after canonicalization, so a block can read naturally in both directions:

`guji.cave`

```
lot/guji-26 IS lot
  SUPPLIED-BY kaffa-coop
  USED-BY coffee/yirgacheffe
```

```
$ cave add --db roastery.db guji.cave
added 3 claim(s), 0 edge(s)

$ cave query --db roastery.db 'kaffa-coop SUPPLIES ?lot'
?lot = lot/yirgacheffe-26
?lot = lot/guji-26

$ cave query --db roastery.db 'coffee/yirgacheffe USES ?lot'
?lot = lot/yirgacheffe-26
?lot = lot/guji-26
```

`SUPPLIED-BY kaffa-coop` under `lot/guji-26` became `kaffa-coop SUPPLIES lot/guji-26`, which is why the reverse declaration matters: without it a bare `SUPPLIED-BY` continuation would have no way to know which end to inherit. Continuation is sugar for writing siblings; it does not qualify the parent and creates no edge.

## 6.3 Grouped claims

An indented line that is a complete claim, subject and all, is simply a claim that happens to be grouped under its parent for readability:

```
resting PRECEDES packing
  packing PRECEDES delivery
```

This is the same as writing the two lines at the margin. Use it to keep related facts visually together in a file; the store does not care.

## 6.4 Explicit full claims

Some entity names collide with the words that introduce line structure. A leading `@claim` says that the following tokens form a full claim:

```
@claim WHEN EXISTS
  @claim IS EXISTS
  WHEN @claim NOT EXISTS
```

The subjects here are the entities `WHEN`, `IS`, and `NOT`. The marker does not turn them into quoted literals or add a context. A trailing `@claim` still means the ordinary context `claim`. Existing unmarked continuations and qualifiers keep their meaning. Canonical export adds the marker where needed; readers of that output must support explicit full claims. An incomplete explicit claim can also serve as a prefix header.

## 6.5 Prefix headers

The fourth form is the one the roastery file uses most: an **incomplete** line followed by indented lines. The incomplete line is a prefix, and every child completes it:

```
lot/huila-26 HAS
  process: natural
  price: 7.80 USD/kg
  score: 84 @src:cupping/june
```

This is three claims, each beginning `lot/huila-26 HAS`. The header is not a claim and is not stored; a trailing comment on a header, or a comment block directly above it, is documentation only, while a comment block directly above a leaf belongs to that leaf's claim. Prefixes nest, and blank lines and comment lines inside them are transparent:

```
lot/huila-26 HAS
  price:
    7.80 USD/kg @2026-Q2
    8.10 USD/kg @2026-Q3
  process: natural
```

The rule that keeps this unambiguous is that only an **incomplete** line can be a prefix. As soon as the accumulated line is a complete claim, its indented children are qualifiers, continuations, or grouped claims as above. `cave parse` counts headers separately, which is what the `prefix` figure in its summary means.

## 6.6 What canonical output looks like

`cave export` prints a store as canonical text, and canonical text uses the same four forms. Adjacent sibling claims are factored through their shared prefix, qualifier edges are rendered as indented qualifier lines, and everything else is one claim per line:

```
$ cave export --db roastery.db | sed -n '/^coffee\/morning-blend HAS defect/,/^stale-lot/p'
coffee/morning-blend HAS defect: flat @cafe/harbour @src:cli @ 70%
  WHEN grinder/harbour HAS burr-age: 14mo @src:cli
  BECAUSE cupping/july @src:cli
stale-lot CAUSE flat-taste @src:cli @ 50%
```

The exported text is the interchange format: `cave import` reads it back, edges included, and produces an equivalent store. The store is a graph; the text is a tree-shaped projection of it, and where a shared row is cited by several parents the export simply restates it (Chapter 24 covers the exact rules).

> **In short.** An indented `WHEN`/`UNLESS`/`VIA`/`BECAUSE` line is a qualifier and creates an edge. An indented bare verb is a continuation that inherits the parent's subject (in object position for an inverse verb). An indented full claim is grouped, nothing more. An incomplete line followed by children is a prefix header that is expanded, not stored.

PART

# The store

What happens to a claim once it is recorded: belief that only ever grows, where each claim came from, how to ask questions, time in the world, disagreement, names that mean the same thing, and what a healthy store looks like.

CHAPTER 7

# Belief That Only Grows

A CAVE store never edits a row. Every change of mind is a new line, and the old line stays. This chapter explains what that buys you, what a **claim key** is, how the store decides what it currently believes, and how to look back at what it believed before.

## 7.1 Append, never update

Load the roastery and record that la-cima raised the price of the Huila lot:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ echo 'lot/huila-26 HAS price: 8.20 USD/kg ; la-cima raised it in August' | cave add --db
roastery.db
added 1 claim(s), 0 edge(s)

$ cave query --db roastery.db 'lot/huila-26 HAS price: ?p'
?p = 8.20 USD/kg  ; la-cima raised it in August

$ cave query --db roastery.db 'lot/huila-26 HAS price: ?p' --all
?p = 7.80 USD/kg
?p = 8.20 USD/kg  ; la-cima raised it in August
```

The query answers with the new price, and `--all` shows that the old one is still there. Nothing was overwritten. The store now knows two things: what the price is, and that it used to be something else.

## 7.2 Claim keys and current belief

How did the store know that the second line was about the **same** fact as the first, rather than a second, contradictory price? Every claim has a **claim key**, computed from the parts of the claim that identify the question it answers:

- for a relation, the subject, verb, object, whether it is negated, and its contexts;
- for an attribute, the subject, verb, attribute **name**, and its contexts.

The value of an attribute is deliberately outside the key. `lot/huila-26 HAS price: 7.80 USD/kg` and `lot/huila-26 HAS price: 8.20 USD/kg` answer the same question, "what is the price", so they belong to one **belief series**. Within a series, the row with the newest transaction is the **current belief**, and that is what queries return by default.

Contexts are inside the key on purpose. `lot/huila-26 HAS score: 84 @src:cupping/june` and `lot/huila-26 HAS score: 86.5 @src:q-grader/ana` are different series: two sources answering the same question are two voices, and the store keeps both until you ask it to choose (Chapter 11). Keys are computed after a claim is normalized to its primary direction, which is why a reverse-verb spelling lands in the same series as the forward one (Chapter 3).

## 7.3 Retraction

To withdraw a claim, append it again at zero confidence. The series then has a current row that supports nothing:

```
$ echo 'la-cima SUPPLIES lot/santa-ana-26 @ 0% ; sold out before we committed' | cave add --
db roastery.db
```

```
added 1 claim(s), 0 edge(s)

$ cave query --db roastery.db 'la-cima SUPPLIES ?lot'
?lot = lot/huila-26

$ cave query --db roastery.db 'la-cima SUPPLIES ?lot' --all
?lot = lot/huila-26
?lot = lot/santa-ana-26
?lot = lot/santa-ana-26  ; sold out before we committed
```

The retracted fact disappears from ordinary reads and stays in the history. Compare this with negation. `cafe/harbour STOCKS NOT coffee/yirgacheffe` asserts something: they do not stock it, at whatever confidence you give the line. `cafe/harbour STOCKS coffee/yirgacheffe @ 0%` merely says the positive claim has no support any more. Retract when a claim was wrong or has stopped being true; negate when the negative is itself a fact worth knowing.

## 7.4 Contradictions are allowed

A store accepts claims that contradict each other. Two graders can score one lot differently; two sources can disagree on a supplier's country; a claim and its negation can both be current, from different sources. CAVE treats disagreement as data. The alternative, rejecting the second claim at write time, would throw away exactly the knowledge that most needs recording: that sources disagree, and about what.

What the store promises instead is that every claim is attributable and every read is explicit about what it returns. Default reads return all current beliefs, disagreements included. Filtered reads (`WHERE conf >= 0.8`), resolved reads (`--resolve`, Chapter 11), and time-anchored reads (`--as-of` below, `--at` in Chapter 10) narrow that in ways you choose.

## 7.5 The history, as text

`cave export` prints the store as canonical CAVE text, oldest row first, with the belief series visible. `--current` prints only current beliefs:

```
$ cave export --db roastery.db | grep 'huila-26 HAS price'
lot/huila-26 HAS price: 8.20 USD/kg @src:cli ; la-cima raised it in August

$ cave export --db roastery.db | tail -n 2
lot/huila-26 HAS price: 8.20 USD/kg @src:cli ; la-cima raised it in August
la-cima SUPPLIES lot/santa-ana-26 @src:cli @ 0% ; sold out before we committed
```

The first price sits inside the factored `lot/huila-26 HAS` block earlier in the export; the two rows appended in this chapter come after everything from the original file, in the order they were recorded. Importing that text into a fresh store replays it in order, so the same rows become current again. This is why the export is the interchange format and the store file is a working index: the text carries the history (Chapter 24 adds transaction identity to it).

## 7.6 Looking back: `--as-of`

Because rows are never removed, the store can answer any question **as it would have answered it at an earlier moment**. `--as-of` takes a date, a timestamp, or a transaction id, and hides every row recorded after that boundary.

Each row's transaction id is a UUIDv7, which encodes when it was recorded. The annotated export shows the id above each claim; here we pick the id of the original price and ask what the price was as of that append:

```
$ cave export --db roastery.db --tx --max-sensitivity restricted \
    | grep -B1 'price: 7.80' | grep -o '[0-9a-f-]\{36\}' > before.tx
```

```
$ cave query --db roastery.db 'lot/huila-26 HAS price: ?p' --as-of "$(cat before.tx)"
?p = 7.80 USD/kg
```

A date form, `--as-of 2026-08-15`, includes the whole day; a timestamp, `--as-of 2026-08-15T09:00:00Z`, includes that second. Everything the engine resolves moves to the same instant: alias links, verb renames, and the transitive hops of a query are all computed from what was believed then. No snapshots are involved; the answer is reconstructed from rows that were always there.

**Permanence includes mistakes**
There is no claim-level delete. A retraction changes current belief; it does not erase the earlier row from the store, its exports, its backups, or any store it was synced to. Treat everything you record as permanent, and keep credentials and data that must be selectively erased out of a store. Chapter 8 has the details.

> **In short.** Every change is an append. A claim key identifies the question a claim answers; the newest row per key is current belief, and `--all` shows the rest. Retract with `@ 0%`; negate with `NOT`. Contradictions coexist and are resolved at read time. `--as-of` reconstructs belief at any past moment from the rows that are still there.

CHAPTER 8

# Provenance

A store that only ever grows is only as useful as its answers to "says who?" This chapter is about how claims are attributed: the stamp every append surface adds, the four dimensions the store indexes behind the compact context syntax, line-level citations into sources, audience labels, and the exact backup that preserves all of it.

## 8.1 Every append is stamped

Load the roastery and look at the exported text:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ cave export --db roastery.db | sed -n '/^kaffa-coop IS/,/^la-cima HAS/p'
kaffa-coop IS supplier @src:cli
la-cima IS supplier @src:cli
kaffa-coop HAS country: ethiopia @src:cli
la-cima HAS country: colombia @src:cli
```

None of those lines carried `@src:cli` in the file. The command added it, because `cave add` is the interactive surface and a claim that names no source is stamped with the surface that appended it. Each surface has its own stamp:

| Surface | Stamp |
|---|---|
| `cave add` | `@src:cli` |
| an agent over MCP | `@src:agent/<client-name>` |
| `cave ingest` in stdout mode | `@src:ingest` |
| `cave connect` | `@src:connect/<name>/<key>`, one per record |
| a rule conclusion | `@src:rule/<digest>` |
| an action effect | `@src:action/<name>` |
| an automation's agent reply | `@src:automation/<name>` |

A claim that already names a source, like the cupping scores with `@src:cupping/june`, is left alone: authored provenance wins over the stamp. The exceptions are the **lifecycle** stamps of connect, rules, and actions, which are added even when the claim names its own source, because the engine must be able to find its own output later to retract or supersede it.

The stamp is applied **before** the claim key is computed. Since contexts are part of the key (Chapter 7), the same fact asserted by two actors lives in two series. That is deliberate: an agent restating a human's claim does not overwrite it, and a human correcting an agent's claim does not silently disappear on the next agent run. To write into another actor's series on purpose, name its context explicitly:

```
lot/huila-26 HAS process: washed @src:agent/claude @ 0% ; wrong, it is a natural
```

`cave import` never stamps, because it replays exported text and replayed claims must keep the keys they were exported with. `cave add --no-src` opts out for a single run.

## 8.2 Four dimensions behind one syntax

The context syntax stays compact, but the store does not treat every `@src:` the same way. Each row also has an indexed **provenance projection** with four dimensions:

| Dimension | Meaning |
|---|---|
| actor | the surface or agent that appended the row |
| source | the physical evidence locator, without any line fragment |
| run | the engine-owned lifecycle identity a generated row belongs to |
| domain | an explicit `@scope:<name>` partition |

Connect, derive, act, and automate find their own rows by the **run** dimension, never by searching for a context string, so an authored `@src:` that happens to look like an engine stamp cannot escape or spoof engine ownership. Ordinary queries, claim keys, and exports see only the contexts. Opening an older store backfills the projection conservatively.

## 8.3 Citing the exact line

A source context can point at the line or line range that supports a claim. The fragment is one-based and inclusive:

```
lot/huila-26 HAS moisture: 11.4% @src:emails/la-cima-2026-08-12.txt#L14
lot/huila-26 HAS eta: 2026-09-03 @src:emails/la-cima-2026-08-12.txt#L14-L16
```

The whole context is still part of the claim key and travels through export, import, and sync unchanged. The decoded locator without the fragment is the source's identity for resolution policy (Chapter 11), so two spans from one email are one source for trust purposes and two distinct claim series for history. Reserved characters in the locator are percent-escaped: a space is `%20`, a literal `#` is `%23`.

`cave ingest` numbers the text it hands to a model and asks for the smallest supporting range (Chapter 15); `cave connect` records the exact CSV line of every mapped record (Chapter 14); reports render the location as a footnote and link it when the source is a URL (Chapter 21).

> **A span is a pointer, not an archive**
> The line range points into the version of the source that was cited. If the evidence must stay immutable, keep or version the source itself; the store keeps the pointer, not the page.

## 8.4 Audience labels

Not every claim is for every reader. A sensitivity tag labels a row's audience, from `public` through `internal` and `confidential` to `restricted`. Unlabeled rows are `internal`; a malformed label fails closed as `restricted`.

```
$ echo 'la-cima HAS contract-price: 7.10 USD/kg #sensitivity:confidential' | cave add --db
roastery.db
added 1 claim(s), 0 edge(s)

$ cave export --db roastery.db | grep contract-price
[exit 1]

$ cave export --db roastery.db --max-sensitivity confidential | grep contract-price
la-cima HAS contract-price: 7.10 USD/kg @src:cli #sensitivity:confidential
```

The publication surfaces, `cave export`, `cave serve`, and `cave report`, default to an `internal` ceiling and must be told explicitly to go higher. Filtering is structural: counts, search, history, and lineage are computed from visible rows only, and an edge is emitted only when both endpoints are visible. For a current-only export, current belief is resolved over the complete history first and the selected row is then filtered, so a hidden newer row never revives an older public belief there; the served page and reports query a store already

narrowed to the ceiling. Local queries and the general MCP tools are not narrowed; when their output will be published, route it through a scoped surface. Labels are routing metadata, not encryption or access control.

## 8.5 What permanence means

Retraction and `--current` change what is believed; they erase nothing. A row remains in the store, in every export that includes history, in every peer it was synced to, and in every backup. CAVE deliberately provides no claim-level delete, because a local delete could not make an honest promise across SQLite free pages, full-text index shadow tables, transaction journals, copies, version control, and other machines; and an older peer could reintroduce the row under its global id anyway.

The consequence is a rule for what to record: nothing that must be selectively erased later. Credentials, private keys, and personal data with a retention policy stay out of the store, out of comments, and out of source citations. If a secret is ingested by accident, rotate it first, stop writers and sync, inventory every copy, rebuild a store from reviewed safe input, and then destroy the affected copies with your storage provider's confirmed procedure. Never merge an affected store back into the clean one.

## 8.6 Exact backups

Canonical text is portable, but importing it mints fresh transaction ids. For an exact copy, with every id, transaction, edge, and provenance row, `cave backup` snapshots the live store:

```
$ cave backup --db roastery.db --out roastery.snapshot.db
created exact backup (<n> row(s), schema v2, <n> bytes, sha256:<hex>) at <path>

$ cave backup --verify roastery.snapshot.db
verified exact backup (<n> row(s), schema v2, <n> bytes, sha256:<hex>) at <path>

$ cave restore roastery.snapshot.db --db restored.db
restored exact backup (<n> row(s), schema v2, <n> bytes, sha256:<hex>) at <path>

$ cave query --db restored.db 'la-cima SUPPLIES ?lot'
?lot = lot/huila-26
?lot = lot/santa-ana-26
```

The snapshot is taken online with SQLite's `VACUUM INTO`, so readers and writers can keep going. It is written to a temporary file, checked for integrity and schema, hashed, and published atomically; the hash printed is the token you record and pass to `--sha256` on verify and restore. Restore refuses a destination with WAL or journal sidecars, because that means something is still using it: stop every process first. On any failure the previous destination is untouched. Snapshots from schema version 1 onward remain recoverable through the current version. Verify and restore preserve their bytes; opening an older restored copy for writing migrates that copy, leaving the retained backup unchanged.

> **In short.** Every append surface stamps `@src:` unless the claim names its own source, and the stamp is part of the key, so actors get separate series. Behind the syntax the store indexes actor, source, run, and domain. `@src:file#L10-L12` cites lines. `#sensitivity:<level>` labels audience and the publication surfaces filter to a ceiling. Nothing is ever deleted; keep secrets out. `cave backup` and `cave restore` preserve exact identity.

CHAPTER 9

# Asking Questions

CAVE-Q is the query language, and there is almost nothing to learn: a query is a claim with holes in it. This chapter walks through variables, wildcards, filters, reverse patterns and paths, joined patterns, paging, and the JSON form, and ends with the SQL escape hatch.

## 9.1 Patterns

A pattern is a claim in which some slots are `?variables`. The store finds every current claim that fits and prints the bindings:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ cave query --db roastery.db '?cafe STOCKS coffee/morning-blend'
?cafe = cafe/north
?cafe = cafe/harbour

$ cave query --db roastery.db '?lot HAS process: ?how'
?lot = lot/yirgacheffe-26  ?how = washed
?lot = lot/huila-26  ?how = natural
?lot = lot/santa-ana-26  ?how = washed
```

A variable can stand in subject, object, or value position. The attribute name is not a variable slot; ask for a specific attribute. Two or more variables bind together, one line per solution. When the matched claim carries a comment, the binding line ends with it after `;`, so the note an author left next to a claim travels with the answer:

```
$ cave query --db roastery.db '?lot HAS score: ?s'
?lot = lot/yirgacheffe-26  ?s = 87
?lot = lot/huila-26  ?s = 84
?lot = lot/santa-ana-26  ?s = 85  ; clean, but nothing remarkable
```

A pattern with no variables asks whether the claim holds, and prints the matching rows as they were written. An underscore is an anonymous wildcard for a slot you do not care about:

```
$ cave query --db roastery.db 'la-cima SUPPLIES lot/huila-26'
la-cima SUPPLIES lot/huila-26

$ cave query --db roastery.db '_ SUPPLIES lot/huila-26'
la-cima SUPPLIES lot/huila-26
```

Metadata in a pattern narrows it. A tag or a context must be present on the claim; `NOT` matches only explicitly negated claims:

```
$ cave query --db roastery.db '?lot HAS score: ?s @src:cupping/june'
?lot = lot/yirgacheffe-26  ?s = 87
?lot = lot/huila-26  ?s = 84
?lot = lot/santa-ana-26  ?s = 85  ; clean, but nothing remarkable
```

There is no negation-as-failure. “Lots that have no score” is not a pattern; it is a health check (Chapter 13).

## 9.2 Filters

A second argument beginning with `WHERE` filters the matches on stored fields: confidence, a value with its unit, or transaction time.

```
$ cave query --db roastery.db '?lot HAS price: ?p' 'WHERE value <= 8.10 USD/kg'
?lot = lot/huila-26  ?p = 7.80 USD/kg
?lot = lot/santa-ana-26  ?p = 8.10 USD/kg

$ cave query --db roastery.db '?lot HAS score: ?s' 'WHERE conf >= 0.9'
?lot = lot/yirgacheffe-26  ?s = 87
?lot = lot/huila-26  ?s = 84
?lot = lot/santa-ana-26  ?s = 85  ; clean, but nothing remarkable
```

`WHERE tx <= 2026-08-01` restricts by when a row was recorded; a date means the whole UTC day. Timestamps without an offset are UTC on every host.

## 9.3 Reading backwards

An inverse verb is a valid pattern verb and compiles to the same physical query as the primary direction (Chapter 3). The two questions below hit the same rows:

```
$ cave query --db roastery.db 'coffee/morning-blend USES ?lot'
?lot = lot/huila-26
?lot = lot/santa-ana-26

$ cave query --db roastery.db 'lot/huila-26 USED-BY ?coffee'
?coffee = coffee/morning-blend
```

## 9.4 Following a chain

A + after the verb follows it for as many hops as it takes. This is how a question nobody wrote down as a single claim gets answered. Add a small geography to the store:

`regions.cave`

```
; where the lots come from, as a containment tree

colombia CONTAINS region/huila
region/huila CONTAINS lot/huila-26
region/huila CONTAINS lot/santa-ana-26
ethiopia CONTAINS region/gedeo
region/gedeo CONTAINS lot/yirgacheffe-26
```

```
$ cave add --db roastery.db regions.cave
added 5 claim(s), 0 edge(s)

$ cave query --db roastery.db '?what PART-OF+ colombia'
?what = lot/huila-26
?what = lot/santa-ana-26
?what = region/huila

$ cave query --db roastery.db 'lot/yirgacheffe-26 PART-OF+ ?where'
?where = ethiopia
?where = region/gedeo
```

Transitive matches are computed over current positive edges. They carry no single row, which is why they print bindings and never a raw line, and why they cannot be anchored in valid time (Chapter 10). The classic use is a dependency chain: `?x USES+ core` answers "what breaks if `core` changes".

A `*` instead of `+` also counts the zero-hop path, so the starting point answers too. Parentheses list several verbs, and the walk may take any of them at each step, so a cafe reaches the lots in its cups through the coffees it stocks:

```
$ cave query --db roastery.db 'region/huila CONTAINS* ?x'
?x = lot/huila-26
?x = lot/santa-ana-26
?x = region/huila

$ cave query --db roastery.db 'cafe/north (STOCKS|USES)+ ?what'
?what = coffee/morning-blend
?what = coffee/yirgacheffe
?what = lot/huila-26
?what = lot/santa-ana-26
?what = lot/yirgacheffe-26
```

An inverse verb in the list is walked backwards, so listing both `CONTAINS` and `PART-OF` walks the containment tree in both directions. Without a `+` or `*`, a list such as `(STOCKS|USES)` matches one claim of either verb and prints its rows like any single-hop pattern.

## 9.5 Joining patterns

A question that spans several claims is several patterns separated by commas. They are read left to right, and a variable that appears twice must take the same value in both places. This is exactly the left-hand side of a rule (Chapter 17) without the conclusion, and it accepts the same `?var < value` constraints:

```
$ cave query --db roastery.db '?cafe STOCKS ?coffee, ?coffee USES ?lot, ?lot HAS process:
washed'
?cafe = cafe/north  ?coffee = coffee/morning-blend  ?lot = lot/santa-ana-26
?cafe = cafe/north  ?coffee = coffee/yirgacheffe  ?lot = lot/yirgacheffe-26
?cafe = cafe/harbour  ?coffee = coffee/morning-blend  ?lot = lot/santa-ana-26

$ cave query --db roastery.db '?lot HAS price: ?p, ?p < 9 USD/kg'
?lot = lot/huila-26  ?p = 7.80 USD/kg
?lot = lot/santa-ana-26  ?p = 8.10 USD/kg
```

Further arguments shape the answer, one clause each. `SELECT` keeps only the variables you name, and `SELECT DISTINCT` also folds repeated answers together. `ORDER BY` sorts: numbers numerically, text alphabetically, with `DESC` for descending order:

```
$ cave query --db roastery.db '?cafe STOCKS ?coffee, ?coffee USES ?lot, ?lot HAS process:
washed' 'SELECT DISTINCT ?cafe'
?cafe = cafe/north
?cafe = cafe/harbour

$ cave query --db roastery.db '?lot HAS score: ?s' 'ORDER BY ?s DESC'
?lot = lot/yirgacheffe-26  ?s = 87
?lot = lot/santa-ana-26  ?s = 85
?lot = lot/huila-26  ?s = 84
```

`OPTIONAL` adds what is known without demanding it. Each solution of the lines above keeps its place. If the optional premises can extend it, they do; if they cannot, its optional variables stay unbound. An unbound variable means “not recorded”, not “false”, and it sorts last:

```
$ cave query --db roastery.db '?lot IS lot' 'OPTIONAL ?lot HAS score: ?s, ?s >= 85' 'ORDER
BY ?s DESC'
?lot = lot/yirgacheffe-26  ?s = 87
?lot = lot/santa-ana-26  ?s = 85
?lot = lot/huila-26
```

A `WHERE` line still filters the one pattern directly above it. A solution stands on several claims, so it has no single comment to print; in JSON the claims a solution stands on arrive as its `support`. There is still no negation-as-failure: `OPTIONAL` reports what is missing, and it never treats missing as false.

## 9.6 Current, all, and the rest

By default a query sees current beliefs: the newest row per claim key, positive, not retracted. `--all` sees every row instead, which is how you read history. Three more switches change which rows are in play, and each has its own chapter: `--as-of` reconstructs belief at a past moment (Chapter 7), `--at` anchors in valid time (Chapter 10), `--resolve` keeps only the winners among contested facts (Chapter 11), and `--aliases` widens names through alias links (Chapter 12). They compose: a resolved, alias-aware read as of last month is one command. Historical vocabulary also excludes later qualifier parents: attaching one today cannot hide yesterday's declaration.

## 9.7 Paging and JSON

A query page holds a hundred matches by default and at most a thousand. When there are more, the human output ends with a `next:` token, and `--cursor` continues the same frozen snapshot. Empty stores and historical snapshots still validate query syntax and time options; invalid requests fail rather than returning an empty page:

```
$ cave query --db roastery.db '?lot IS lot' --limit 2
?lot = lot/yirgacheffe-26
?lot = lot/huila-26
next: <token>
```

If sync adds older rows or lineage touching them, the cursor fails with a restart instruction. Start again without `--cursor`; later local appends and wholly future lineage remain compatible. Each page checks for historical changes before and after reading, so a concurrent change cannot mix its results.

`--json` emits a versioned `cave.query-page` object whose matches carry the bindings and the full claim record, including its canonical line, id, transaction, and claim key. It is the form to use from a program:

```
$ cave query --db roastery.db '?who SUPPLIES lot/huila-26' --json | head -n 11
{
  "format": "cave.query-page",
  "version": 1,
  "snapshot": "<uuid>",
  "matches": [
    {
      "format": "cave.query-match",
      "version": 1,
      "bindings": {
        "who": "la-cima"
      },
```

## 9.8 Words, not patterns

A pattern needs a name to start from. When all you remember is wording, a phrase from a comment, a value, a tag, `cave search` runs the store's full-text index (no extra service to run) over every claim: subject, verb, object, attribute name, value text, comment, and the line as written, so tags, contexts, and inverse spellings match too. The terms are one literal phrase, and matches print newest first as raw lines, comments included:

```
$ cave search --db roastery.db 'nothing remarkable'
lot/santa-ana-26 HAS score: 85 @src:cupping/june ; clean, but nothing remarkable

$ cave search --db roastery.db washed
```

```
lot/santa-ana-26 HAS process: washed
lot/yirgacheffe-26 HAS process: washed
```

`--raw` passes FTS5 `MATCH` syntax through: `AND`, `OR`, `NOT`, `NEAR`, `prefix*`, and column filters such as `comment:heap`. Words split on punctuation, so `heap-dump` and "heap dump" are the same phrase, and `auth/middleware` matches `auth middleware`:

```
$ cave search --db roastery.db --raw 'cupping NOT remarkable'
lot/huila-26 HAS score: 84 @src:cupping/june
lot/yirgacheffe-26 HAS score: 87 @src:cupping/june
```

Search reads the whole history, superseded and retracted rows included, because the index exists to find the wording, not to judge it; once it has named the entity, a query answers what is currently believed. `--limit` caps the matches (a hundred by default) and `--json` returns the full claim records.

## 9.9 A file is a store

`--db` names a store by what is in the file, not by the file's name. A SQLite file opens as usual; any other file is read as CAVE text and replayed into an in-memory store before the command runs, holding exactly the claims `cave import` would have stored, plus whatever sources the file declares (Chapter 14). The file this chapter started from answers questions without ever having been added anywhere:

```
$ cave query --db roastery.cave '?lot HAS process: ?how'
?lot = lot/yirgacheffe-26  ?how = washed
?lot = lot/huila-26  ?how = natural
?lot = lot/santa-ana-26  ?how = washed
```

Nothing is written. A command that appends refuses a text file and points at `cave import`, and a read against a path that does not exist is an error rather than a fresh, empty database, so a typo in `--db` never looks like a store with nothing in it:

```
$ printf 'lot/huila-26 HAS process: honey\n' | cave add --db roastery.cave
cave add: roastery.cave is CAVE text, not a database — text stores are read-only;
materialize it with `cave import --db <store.db> roastery.cave`
[exit 1]

$ cave query --db roastry.db '?lot HAS process: ?how'
cave query: no store at roastry.db — create one with `cave add --db roastry.db`, or pass a
CAVE text file
[exit 1]
```

## 9.10 SQL, when you need it

Today's store is an ordinary SQLite file with a documented schema (Chapter 25), and nothing stops you from opening it. CAVE-Q is the ergonomic layer for graph questions; SQL is the transparent escape hatch for analysis the pattern language does not cover. The current-belief query is one join, and it is worth knowing so that hand-written SQL agrees with the tool:

```
SELECT c.* FROM cave_claim c
JOIN (SELECT claim_key, MAX(tx) AS max_tx FROM cave_claim GROUP BY claim_key) latest
  ON c.claim_key = latest.claim_key AND c.tx = latest.max_tx
WHERE c.conf > 0 AND c.negated = 0;
```

> **In short.** A query is a claim with `?variables`; `_` is a wildcard; a fully-bound pattern prints the matching rows. Tags and contexts in the pattern narrow it, `WHERE` filters on confidence, value, and transaction time. Inverse verbs query the same rows; `VERB+` follows a chain. Default reads are current belief; `--all`, `--as-of`, `--at`, `--resolve`, and `--aliases` change the universe and compose. A CAVE text file is a store for every command that only reads. `--json` for programs, SQL for everything else.

CHAPTER 10

# Time in the World

Chapter 7 dealt with **transaction time**: when the store learned something. This chapter adds the other axis, **valid time**: when a claim holds in the world. The two are independent, and keeping them apart is what lets one store answer "what did we believe in June about the September price".

## 10.1 Time contexts

A context whose body looks like a date names a period, and the engine reads it as a time context. Points name whole calendar periods; ranges join two points with `..` and may leave either end open:

| Context | Covers |
|---|---|
| `@2026` | the whole year |
| `@2026-08` | the month |
| `@2026-08-14` | the day |
| `@2026-Q3` | July to September |
| `@2026-H2` | July to December |
| `@2026-W33` | the ISO week |
| `@2026-Q2..2026-Q4` | April through December, whole periods at both ends |
| `@2026..` | from the start of 2026, open-ended |
| `@..2025` | through the end of 2025 |

A time context is an ordinary context in every other respect: it is stored, exported verbatim, and part of the claim key. A context that does not parse as a time is simply opaque text, never an error. `@time:2026-Q3` is the same time context with the recommended prefix.

## 10.2 Anchoring a query: `--at`

`--at` anchors a query at an instant. A claim applies at that instant when it has no time context at all (timeless knowledge, which is most of it) or when one of its time contexts covers it. Everything else is invisible:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ printf '%s\n' \
    'kaffa-coop SUPPLIES lot/guji-26 @2026-Q3..2026-Q4 ; a one-off, this season only' \
    'roastery HAS headcount: 3 @..2025' \
    'roastery HAS headcount: 4 @2026..' \
    | cave add --db roastery.db
added 3 claim(s), 0 edge(s)

$ cave query --db roastery.db 'kaffa-coop SUPPLIES ?lot' --at 2026-08
?lot = lot/yirgacheffe-26
?lot = lot/guji-26  ; a one-off, this season only

$ cave query --db roastery.db 'kaffa-coop SUPPLIES ?lot' --at 2026-02
?lot = lot/yirgacheffe-26

$ cave query --db roastery.db 'roastery HAS headcount: ?n' --at 2025-06
?n = 3
```

```
$ cave query --db roastery.db 'roastery HAS headcount: ?n' --at 2026-06
?n = 4
```

The timeless `lot/yirgacheffe-26` relation matches at both anchors; the seasonal `lot/guji-26` relation matches only inside its range. The two headcount claims are a **step function**: consecutive scalar claims whose ranges tile, with no boundary period repeated, so that exactly one applies at any instant. Without `--at`, both are current beliefs and both are returned; the anchor is what turns a pile of dated claims into one answer.

A point anchor is read as the **start** of its period, so `--at 2026` is the first instant of the year; name a finer period to land inside one. An anchor that does not parse is an error, not an empty result.

## 10.3 Trajectories: `A -> B`

Some values move. A trajectory is a value with two numeric endpoints and one unit, and it interpolates linearly across the claim's single closed range:

```
$ echo 'lot/huila-26 HAS price: 7.80 -> 8.60 USD/kg @2026..2027 ; la-cima indexed the
contract' | cave add --db roastery.db
added 1 claim(s), 0 edge(s)

$ cave query --db roastery.db 'lot/huila-26 HAS price: ?p' --at 2026-07
?p = 7.80 USD/kg
?p = 8.197 USD/kg  ; la-cima indexed the contract

$ cave query --db roastery.db 'lot/huila-26 HAS price: ?p' --at 2027-06
?p = 7.80 USD/kg
?p = 8.6 USD/kg  ; la-cima indexed the contract
```

The first binding is the timeless price from the roastery file, which always applies. The second is the trajectory evaluated at the anchor: 7.80 at the start of 2026, 8.60 at the start of 2027, linear in between, and holding at the end value through the end period. A value-slot variable binds the interpolated number, while the stored row is returned untouched.

A trajectory is deliberately not one number: its numeric column is empty, so `WHERE value` filters never match it and fusion skips it. It evaluates only under `--at`, and only when the claim has exactly one closed range; an open range or several ranges leave it textual. Endpoints with different units do not form a trajectory at all. Re-estimating a trajectory under the same contexts appends to the same belief series, so `--as-of` reconstructs the earlier estimate.

## 10.4 Both axes at once

`--as-of` chooses which rows are **believed**; `--at` chooses **when in the world** they apply. They compose, and so do `--all`, `--aliases`, and `--resolve`: a universe of rows is fixed first, then the valid-time pass runs over it. The bitemporal question, "what did we believe on the first of August about the price in June", is one command with both flags.

Transitive patterns reject `--at`. Hop edges are not valid-time filtered, and a closure that silently ignored the anchor would be a wrong answer rather than an approximate one.

> **Why no formulas**
> Earlier drafts sketched values as functions of time, `(t -> 20B * 1.25^(t - 2025))`. Linear trajectories and tiled step claims cover the ordinary cases; anything nonlinear belongs in a bounded external evaluator whose inputs, version, and outputs are themselves recorded as claims. A store does not execute a lambda stored as knowledge. See spec §17.5 and §32.

> **In short.** A date-like context (`@2026-Q3`, `@2026..2027`, `@..2025`) is a time context and part of the claim key. `--at <instant>` hides claims whose time contexts miss the instant; timeless claims always match; tiled scalar claims form step functions. `A -> B unit` interpolates across a single closed range under `--at`. `--as-of` and `--at` are orthogonal and compose.

CHAPTER 11

# When Sources Disagree

The store keeps every voice. Sometimes you need one answer anyway: a report wants a single score, an action needs to know whether a precondition holds. This chapter introduces **resolution**, the opt-in read mode that picks a winner per contested fact without rewriting anything, and the policy that decides who wins.

## 11.1 A contest

The June cupping scored the Huila lot at 84. A visiting Q-grader scores it higher. Both claims answer the same question from different sources, so they are different series and both are current:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ echo 'lot/huila-26 HAS score: 86.5 @src:q-grader/ana @ 80%' | cave add --db roastery.db
added 1 claim(s), 0 edge(s)

$ cave query --db roastery.db 'lot/huila-26 HAS score: ?s'
?s = 84
?s = 86.5
```

`cave resolve` lists every fact that more than one series speaks about and ranks the candidates as the policy sees them:

```
$ cave resolve --db roastery.db
lot/huila-26 HAS score: 84 @src:cupping/june ; class 2, effective 100%
  over lot/huila-26 HAS score: 86.5 @src:q-grader/ana @ 80% ; class 2, effective 80%
```

The June score wins because, with nothing else declared, both sources are in the same precedence class and the June claim has the higher confidence. `--resolve` on any query matches only the winners:

```
$ cave query --db roastery.db 'lot/huila-26 HAS score: ?s' --resolve
?s = 84
```

## 11.2 What competes with what

Rows contest one fact when they answer the same question. The **resolution group** is the claim key with every `src:` context removed and the negation flag dropped. So different sources compete, and a claim competes with its own negation; but claims scoped to different non-source contexts, such as `@cafe/north` and `@cafe/harbour`, are different facts and never contest each other. Candidates are the current row of each series in the group, excluding retracted rows: a series at zero confidence neither wins nor blocks, and a group whose candidates are all retracted resolves to unknown.

## 11.3 The policy

Among candidates, the winner is decided by three comparisons in order:

1. **Precedence class.** An integer per source; higher outranks. A row's class is the highest class among its sources; a row with no source takes the root class.
2. **Effective confidence.** Stored confidence times the source's **reliability**, a weight between 0 and 1 that defaults to 1. A row with several sources is as reliable as its weakest one.
3. **Latest transaction.** The tiebreaker, and never a tie itself.

The built-in ladder is what makes a human correction survive an automated re-run: `@src:cli` is class 4, agents and actions are class 3, content sources and connect records are class 2, and rule conclusions are class 1. Recency still decides within a class, and within one series exactly as before.

## 11.4 Declaring policy in-band

Precedence and reliability are knowledge about sources, so they are declared as claims about `source/<name>` entities. Matching is by path prefix, most specific first: `@src:q-grader/ana` takes its reliability from `source/q-grader/ana` if declared, else `source/q-grader`, else the root `source`.

```
$ printf '%s\n' \
    'source/q-grader HAS reliability: 95% ; certified graders' \
    'source/cupping HAS reliability: 60% ; our own table, on a busy morning' \
    | cave add --db roastery.db
added 2 claim(s), 0 edge(s)

$ cave resolve --db roastery.db
lot/huila-26 HAS score: 86.5 @src:q-grader/ana @ 80% ; class 2, effective 76%
  over lot/huila-26 HAS score: 84 @src:cupping/june ; class 2, effective 60%

$ cave resolve --db roastery.db --policy
source           precedence 2
source/action    precedence 3
source/agent     precedence 3
source/cli       precedence 4
source/cupping   reliability 60%
source/q-grader  reliability 95%
source/rule      precedence 1
```

Now the grader's 80 percent, weighted by 95 percent reliability, beats the table's 100 percent weighted by 60. The stored rows are unchanged; reliability ranks, it never rewrites confidence. Policy declarations evolve append-only like everything else, and they themselves resolve under the built-in ladder alone, so an ingested document cannot elevate its own tier above the humans and agents it answers to.

## 11.5 Where resolution applies

Resolution is opt-in everywhere, because the default read must keep disagreement visible. `cave query --resolve` and `cave report --resolve` (Chapter 21) use it; the MCP query and neighbourhood tools accept a `resolve` flag; `--aliases` widens groups through the alias closure (Chapter 12), and `--as-of` reconstructs candidates, policy, and closure at the boundary. `--all` is incompatible with `--resolve`, since it asks for the unresolved history.

For numeric disagreements there is a second option besides picking: fuse. The MCP `cave_fuse` tool combines estimates of one quantity by precision and confidence (Chapter 5), which is often the better answer when the question is "what is the price" rather than "whom do we trust".

**Provenance is claimed, not proven**
A row's sources are whatever contexts it carries. Writing into another actor's series by naming its context lands in that actor's precedence class. The history records exactly who wrote what; resolution trusts the recorded contexts.

> **In short.** Rows contest a fact when they differ only in source or polarity. `cave resolve` ranks candidates by precedence class, then confidence times source reliability, then recency; `--resolve` returns winners only. Declare policy as `source/<name> HAS precedence:` and `HAS reliability:` claims, matched by path prefix. The default read keeps every voice.

CHAPTER 12

# One Entity, Many Names

Chapter 2 asked you to spell every entity the same way everywhere. Real stores do not manage it: a colleague writes `la-cima-coop`, an extraction writes `La Cima`, and the graph quietly splits. This chapter covers `ALIAS`, the closure that lets a query see through it, and the discovery command that finds candidates for you.

## 12.1 Declaring an alias

`ALIAS` asserts that two names denote one entity. It is an ordinary claim, and it is undirected in effect: either spelling of the link is enough.

Suppose later notes used a longer name for the Colombian supplier:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ printf '%s\n' \
    'la-cima-coop HAS contact: "Ana Ruiz"' \
    'la-cima-coop SUPPLIES lot/tolima-26' \
    'lot/tolima-26 IS lot' \
    | cave add --db roastery.db
added 3 claim(s), 0 edge(s)

$ cave query --db roastery.db 'la-cima SUPPLIES ?lot'
?lot = lot/huila-26
?lot = lot/santa-ana-26
```

The new lot is invisible under the old name. Link the names and ask again with `--aliases`:

```
$ echo 'la-cima-coop ALIAS la-cima' | cave add --db roastery.db
added 1 claim(s), 0 edge(s)

$ cave query --db roastery.db 'la-cima SUPPLIES ?lot' --aliases
?lot = lot/huila-26
?lot = lot/santa-ana-26
?lot = lot/tolima-26
```

## 12.2 The closure

Alias-aware reading computes the **alias closure**: the set of names connected by current, positive `ALIAS` claims, read as undirected edges. Matching then widens to every name in the group. Three properties matter:

- **Rows are never rewritten.** The store does not pick a canonical spelling and does not merge anything; it returns the stored spelling. That is why the result above still says `la-cima` on the left and why the new lot's own claim still names `la-cima-coop`.
- **Series stay separate.** Aliased names keep their own belief histories. When they disagree, the closure surfaces both claims rather than hiding one; `cave check` reports such disagreements (Chapter 13).
- **It is opt-in.** Every reader that can widen through aliases does so only when asked: `--aliases` on queries, reports, resolution, derivation, and action preconditions.

The closure applies to entity positions only. Values, attribute names, and verbs are not entities; verb spellings resolve through `RENAMED-TO` instead (Chapter 3). Negated aliases (`a ALIAS NOT b`), retracted aliases, and aliases whose endpoint is a literal never link. Unmerging is a retraction: append `la-cima-coop ALIAS la-cima @ 0%` and the two names fall apart again, both histories intact.

## 12.3 Finding candidates

Naming drift is the entity-resolution bottleneck under extraction, and discovering the pairs is the hard part. `cave suggest-alias` finds candidate pairs using explainable signals and prints `ALIAS` claims at low confidence for review. `--min` accepts a finite score in 0..1 or a percentage; `--limit` requires a positive safe integer. Rebuild the store without the alias to see what it finds:

```
$ cave add --db fresh.db roastery.cave
added 33 claim(s), 0 edge(s)

$ printf '%s\n' 'la-cima-coop HAS contact: "Ana Ruiz"' 'la-cima-coop SUPPLIES lot/tolima-26'
| cave add --db fresh.db
added 2 claim(s), 0 edge(s)

$ cave suggest-alias --db fresh.db
lot/yirgacheffe-26 ALIAS lot/santa-ana-26 #suggested @ 45% ; share process: washed; both IS
lot
la-cima-coop ALIAS la-cima #suggested @ 35% ; segments of la-cima within la-cima-coop; la-
cima prefixes la-cima-coop
```

One suggestion is right and one is wrong, which is the normal shape of the output. The second comes from name similarity: `la-cima` is contained in `la-cima-coop`. The first comes from a shared rare attribute value: exactly two lots are `washed`, and a textual value carried by exactly two entities is treated as a possible identity. The comment carries the evidence so a reviewer can judge it. Suggestions are always between 30 and 50 percent, inside the review band that `cave check` flags, and they are questions, not merges.

Both review moves are ordinary appends. Confirm by asserting the link in your own series; reject by negating it:

```
$ printf '%s\n' 'la-cima-coop ALIAS la-cima ; confirmed' 'lot/yirgacheffe-26 ALIAS NOT lot/
santa-ana-26 ; different lots' | cave add --db fresh.db
added 2 claim(s), 0 edge(s)

$ cave suggest-alias --db fresh.db
no alias suggestions
```

A pair with any `ALIAS` history, positive, negated, or retracted, is never suggested again. The decision sticks, and the store never nags. Signals are deliberately guarded: names that differ only in digits are versions, not drift; numeric values never identify; a value shared by three or more entities is a category, not an identity; and shared relations only strengthen a candidate, never create one, because siblings share parents without being one person.

Suggested lines preserve entity identity. An undirected link may reverse to `42 ALIAS 4-2`, keeping `42` out of the numeric object position. The preferred name in suggestion metadata is unchanged. If neither direction can represent an entity object, discovery reports an error without writing.

`--write` stamps suggestions `@src:suggest/alias`. Under one transaction, it skips pairs with history in either direction, preserving reviews committed while a judge runs. Every fresh line must describe one positive alias relation between its proposed names; a mismatch rejects the whole batch. `--json` returns the selected suggestions; `--json --write` also writes and reports the actual appended count. Alias-aware reads honour written suggestions until reviewed: retract their series using the same direction and source context. An optional `--agent` judges candidates using the ingestion shell contract (Chapter 15).

> **In short.** `a ALIAS b` links two names for one entity; `--aliases` widens matching through the closure without rewriting rows or merging histories. Unmerge by retracting. `cave suggest-alias` proposes candidates from explainable signals at review-band confidence; confirm with a plain `ALIAS`, reject with `ALIAS NOT`, and the pair is never suggested again.

CHAPTER 13

# Shape and Health

A store accepts any claim about anything, and it should: the world does not announce its schema in advance. But once you know what a well-described lot looks like, you want to be told when one is missing a price. This chapter covers `EXPECTS`, the health report, the write gate, and the typed client you can generate from the same declarations.

## 13.1 Expectations are claims

`EXPECTS` declares what instances of a type should carry. The object is an attribute name (lowercase) or a verb (uppercase):

shapes.cave

```
; what a well-described record looks like

lot EXPECTS price #unit:USD/kg
lot EXPECTS score
lot EXPECTS SUPPLIED-BY #cardinality:one
coffee EXPECTS USES
cafe EXPECTS STOCKS
```

An entity is an instance of `lot` when it has a current positive `IS lot` claim, or an `IS` claim into a type that `EXTENDS+` into `lot`. The taxonomy is the only widening mechanism; there are no name globs. Binding follows every reachable type without a depth cutoff. Cycles terminate, and multiple paths do not duplicate an instance check. An attribute expectation is met by a current positive `HAS attr:` claim; a relation expectation by a current positive claim with that verb on the side the verb reads from, so `SUPPLIED-BY` is satisfied by a stored `SUPPLIES` row that names the lot as object.

Two tags narrow an expectation without a second schema language. `#cardinality:one` requires exactly one current match, which matters mainly for relations, since an attribute already has at most one current value per series. `#unit:USD/kg` requires the current value's normalized unit to be exactly that; no conversion is attempted and a unitless value fails. Runtime checks and generated clients reject malformed constraint tags, including unknown cardinalities, repeated values, and units on relation expectations. A gate rolls back newly introduced malformed declarations. Existing malformed declarations need an ungated correction or retraction before checked writes resume.

Expectations evolve like every claim: retract one with `@ 0%`, and a negated declaration documents a deliberate non-expectation. They never constrain writes by default.

## 13.2 The health report

`cave check` reads the store against its own expectations and reports, without writing anything:

```
$ cave add --db roastery.db roastery.cave shapes.cave
added 38 claim(s), 0 edge(s)

$ cave check --db roastery.db
shape: 5 expectation(s), 7 instance(s), 13/13 satisfied
coverage: 38 row(s), 38 fact(s) — 38 current, 0 retracted, 0 negated; avg conf 100%, 0 low
(< 0.3); 16 entities, 9 typed
```

Every lot has a price in the right unit, a score, and exactly one supplier; every coffee uses something; every cafe stocks something. Now a new lot arrives with only a type:

```
$ echo 'lot/tolima-26 IS lot' | cave add --db roastery.db
added 1 claim(s), 0 edge(s)

$ cave check --db roastery.db
shape: 5 expectation(s), 8 instance(s), 13/16 satisfied
violations (3):
  lot/tolima-26 missing attribute price (lot/tolima-26 IS lot; lot EXPECTS price #unit:USD/
kg)
  lot/tolima-26 missing attribute score (lot/tolima-26 IS lot; lot EXPECTS score)
  lot/tolima-26 missing relation SUPPLIED-BY (lot/tolima-26 IS lot; lot EXPECTS SUPPLIED-BY
#cardinality:one)
coverage: 39 row(s), 39 fact(s) — 39 current, 0 retracted, 0 negated; avg conf 100%, 0 low
(< 0.3); 17 entities, 10 typed
[exit 1]
```

The command exits with status 1 while a violation remains, so it works as a CI gate. The other sections are advisory: **stale** current beliefs older than a horizon (`--stale <days>`, default 90; finite and non-negative, with fractional days allowed), **review candidates** with confidence between 30 and 70 percent, and **alias disagreements**, where two names in one alias group carry different current values or opposite polarity for the same fact. Alias groups follow the complete chain of links; long chains do not consume the call stack or require a SQL parameter for every member. The coverage line is the intrinsic measure of a store's quality: how much is typed, how much is retracted, how confident the current beliefs are on average.

## 13.3 The write gate

The same check can run inside an append. `cave add --check` reserves the write transaction before reading baseline violations, then appends, checks, and rolls back if the append **introduced** violations. Concurrent repairs committed before the reservation are part of that baseline:

```
$ echo 'lot/tolima-27 IS lot' | cave add --db roastery.db --check
rejected: 3 new violation(s), nothing added (spec §20.3)
  lot/tolima-27 missing attribute price (lot/tolima-27 IS lot; lot EXPECTS price #unit:USD/
kg)
  lot/tolima-27 missing attribute score (lot/tolima-27 IS lot; lot EXPECTS score)
  lot/tolima-27 missing relation SUPPLIED-BY (lot/tolima-27 IS lot; lot EXPECTS SUPPLIED-BY
#cardinality:one)
[exit 1]
```

Pre-existing violations never block: the gate compares, it does not demand a clean store, so the unfinished `lot/tolima-26` does not stop unrelated progress. Actions (Chapter 18) run inside the same gate by default. One mechanism, two enforcement points.

To satisfy the gate, describe the lot fully in one append:

```
$ printf '%s\n' 'lot/tolima-26 HAS price: 7.95 USD/kg' 'lot/tolima-26 HAS score: 85.5
@src:cupping/august' 'la-cima SUPPLIES lot/tolima-26' | cave add --db roastery.db --check
added 3 claim(s), 0 edge(s)

$ cave check --db roastery.db
shape: 5 expectation(s), 8 instance(s), 16/16 satisfied
coverage: 42 row(s), 42 fact(s) — 42 current, 0 retracted, 0 negated; avg conf 100%, 0 low
(< 0.3); 17 entities, 10 typed
```

## 13.4 A typed client

Applications that want compile-time ergonomics can derive a TypeScript module from the current expectations. CAVE text and CAVE-Q stay the primary interfaces; the generated file is a build artifact:

```
$ cave generate --db roastery.db | head -8
// Generated by CAVE typed-client/v1; do not edit.
// Schema SHA-256: <hex>
import { QuerySql, type Store } from '@cavelang/store'

export const caveClientFormatVersion = 1 as const
export const caveSchemaDigest = "<hex>" as const
export const caveSchema = [
  {
```

Each type becomes an interface and a `read<Type>(store, entity)` function. Attribute readers keep the text, the parsed number, and the unit, with `#unit:` narrowing the unit to a literal type; relation readers honour declared inverses; `#cardinality:one` fields are scalars that throw unless exactly one current row exists, and everything else is a readonly array. The output is sorted by code point and embeds a digest of the normalized schema, so it is byte-stable across declaration order and locale, and generation fails before writing when declarations conflict or a type name cannot map to a TypeScript identifier.

> **In short.** `type EXPECTS attribute` and `type EXPECTS VERB` declare shape as claims, bound through the `IS`/`EXTENDS` taxonomy, with `#cardinality:one` and `#unit:<u>` as the only constraints. `cave check` reports violations (exit 1), stale beliefs, review candidates, alias disagreements, and coverage. `cave add --check` and actions roll back appends that introduce new violations. `cave generate` derives a deterministic typed client.

PART

# Getting knowledge in

Three ways to fill a store without typing: structured records through a template, prose through a language model, and the harness that tells you how good the extraction was.

CHAPTER 14

# Structured Data Without a Model

Inventory lives in a spreadsheet. A CSV row does not need a language model to become three claims; it needs a template. This chapter introduces `cave connect`, which maps records from CSV, TSV, JSON, JSON Lines, SQLite, or a URL through an ordinary CAVE file, deterministically, and keeps the store in step with the source on every re-run.

## 14.1 A template is a CAVE file with holes

The roastery's stock count is a CSV with one row per lot:

`stock.csv`

```
lot,kg,roasted
lot/yirgacheffe-26,42,2026-08-20
lot/huila-26,8,2026-08-18
lot/santa-ana-26,15,2026-08-22
```

The mapping is a CAVE document in which `?field` variables stand for column names:

`stock.map.cave`

```
; one row per lot in the green store

?lot HAS stock: ?kg kg
?lot HAS last-roasted: ?roasted
```

A top-level block that contains a variable is a **record template**, instantiated once per record. A block with no variables is **prelude**: verb declarations and static claims, appended once per run. A record that lacks a field, or whose field is empty, simply drops that line and its children; optional columns yield fewer claims, never malformed ones.

`--dry-run` prints what would be written:

```
$ cave connect stock.csv --map stock.map.cave --dry-run --key lot
; --- prelude

; one row per lot in the green store

; --- record 1

lot/yirgacheffe-26 HAS stock: 42 kg
lot/yirgacheffe-26 HAS last-roasted: 2026-08-20

; --- record 2

lot/huila-26 HAS stock: 8 kg
lot/huila-26 HAS last-roasted: 2026-08-18

; --- record 3

lot/santa-ana-26 HAS stock: 15 kg
lot/santa-ana-26 HAS last-roasted: 2026-08-22
```

Values are formatted by position and never invented. Numbers and booleans render as written; a string that is a safe name inserts verbatim; in payload position a string that parses as a CAVE value (`8.20 USD/kg`, `2026-`

`Q3`, `94.5%`) stays a value; anything else becomes a quoted literal. If you need kebab-case identities, shape them in the source, with a slug column or a `--sql` projection.

## 14.2 Records remember themselves

Run it for real, then run it again:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ cave connect stock.csv --map stock.map.cave --db roastery.db --key lot
connect: 3 record(s): 3 mapped, 0 skipped (unchanged); +6 claim(s)

$ cave connect stock.csv --map stock.map.cave --db roastery.db --key lot
connect: 3 record(s): 0 mapped, 3 skipped (unchanged); +0 claim(s)
  note: prelude unchanged, skipped
```

Each record has a stable identity, `connect/<name>/<key>`, where the name defaults to the file's basename and the key is the `--key` column, or a content digest when no key is given. After a record's claims append, one bookkeeping claim records a digest of the **instantiated text**, so a record is skipped until its data or its mapping changes. Every claim a record produces is stamped `@src:connect/<name>/<key>`, and for CSV, TSV, and JSON Lines sources also with the exact line it came from:

```
$ cave export --db roastery.db | grep 'stock: 8 kg'
  stock: 8 kg @src:connect/stock/lot-huila-26 @src:stock.csv#L3
```

## 14.3 When the source changes

The Huila lot was roasted again and the count drops. Overwrite the CSV:

`stock.csv`

```
lot,kg,roasted
lot/yirgacheffe-26,42,2026-08-20
lot/huila-26,6,2026-08-25
lot/santa-ana-26,15,2026-08-22
```

```
$ cave connect stock.csv --map stock.map.cave --db roastery.db --key lot
connect: 3 record(s): 1 mapped, 2 skipped (unchanged); +2 claim(s)
  note: prelude unchanged, skipped

$ cave query --db roastery.db 'lot/huila-26 HAS stock: ?kg'
?kg = 6 kg

$ cave query --db roastery.db 'lot/huila-26 HAS stock: ?kg' --all
?kg = 8 kg
?kg = 6 kg
```

Only the changed record was re-mapped, and its attribute claims superseded naturally because the value is outside the claim key. A relation claim the record used to produce and no longer does is retracted explicitly, because the record stamp lets the engine diff a record against its own previous output. With `--prune`, records that vanished from the source entirely are retracted the same way. Vocabulary declarations are never retracted; registry history is additive.

A failed record with a known key keeps its previous claims. If a record loses its key, or an unkeyed record fails to format or ingest, the connector cannot tell which previous record it represents. Valid records still update, but disappearance pruning is skipped with a note. Repair the source and run with `--prune` again; a valid empty source still retracts vanished records.

## 14.4 Continuous and query-time reads

`--watch` keeps the command running and re-maps whenever the file or the mapping changes; per-record digests keep each pass incremental. It is a tail loop on one machine, not a platform, and that is the point: a webhook or socket bridge that owns its own authentication and retry can write a watched file, and the core never grows a resident listener.

`--query` is federation-lite. The mapped claims are appended inside a transaction, the pattern runs over the union of store and source, and the transaction rolls back. Nothing persists, not even digests:

```
$ cave connect stock.csv --map stock.map.cave --db roastery.db --key lot --query '?lot HAS
stock: ?kg'
?lot = lot/yirgacheffe-26  ?kg = 42 kg
?lot = lot/huila-26  ?kg = 6 kg
?lot = lot/santa-ana-26  ?kg = 15 kg
```

The same command reads a SQLite table (`--table` or `--sql`), a JSON document (`--records data.items` to point at the array), or a URL that serves JSON or CSV. Dotted variables like `?address.city` walk nested JSON.

## 14.5 A one-line mapping, and SQL first

A mapping short enough to type can be given inline: a comma-separated list of claim lines, the same convention action effects use. And when the data needs reshaping before it is mapped, `--sql` runs a query over the parsed records first, whatever the file format — the rows land in a temporary SQLite table called `records`, cells exactly as the file had them (so a CSV number is text until you cast it), and the query's rows are what the mapping sees:

```
$ cave connect stock.csv --map '?lot HAS stock: ?kg kg' --key lot --db roastery.db --query
'?lot HAS stock: ?kg'
?lot = lot/yirgacheffe-26  ?kg = 42 kg
?lot = lot/huila-26  ?kg = 6 kg
?lot = lot/santa-ana-26  ?kg = 15 kg

$ cave connect stock.csv --sql 'SELECT lot FROM records WHERE CAST(kg AS REAL) > 10' --map
'?lot IS well-stocked' --key lot --dry-run
; --- record 1

lot/yirgacheffe-26 IS well-stocked

; --- record 2

lot/santa-ana-26 IS well-stocked
```

Projection, filtering, and derived columns belong in SQL before the mapping and in rules (Chapter 17) after it; the mapping itself never computes.

## 14.6 Declaring a source

Typing the same `cave connect` line every morning is the kind of thing a store should remember. It can, and with no new syntax: a source is a few attribute claims on a `source/<name>` entity, the same entity Chapter 11 uses for a source's reliability.

`sources.cave`

```
source/stock HAS path: stock.csv
source/stock HAS map: stock.map.cave
source/stock HAS key: lot
source/stock HAS reliability: 90%
```

`path` declares the source; `map`, `key`, `format`, `delimiter`, `table`, `sql`, and `records` mirror the options. A `.cave` path needs no map: the file is its own template. Paths are relative to the store's directory, so a store and

its sources travel together. Because a CAVE text file is itself a store (Chapter 9), this file already answers questions: opening it follows the sources it declares.

```
$ cave query --db sources.cave 'lot/huila-26 HAS stock: ?kg'
?kg = 6 kg

$ cave connect --db sources.cave --list
stock: stock.csv --map stock.map.cave --key lot
```

Add the same lines to a SQLite store and `cave connect` with no source runs every declared source, `--watch` tails all their files, and `cave query --sources` overlays them without writing. A declared source stamps `@src:stock/<key>` rather than `@src:connect/stock/<key>`, so the stamp names the same entity the reliability sits on and the policy applies to what the source yields.

Changing a connected source's declaration replaces its data in one transaction. If any replacement record fails, retirement and all replacement writes roll back together. The previous data survives, and fixing the source and rerunning retries the complete transition. An unchanged declaration continues to isolate failures one record at a time.

> **In short.** `cave connect <source> --map <template>` instantiates a CAVE document per record; blocks without variables are prelude. Records are identified by `--key`, digested, stamped `@src:connect/<name>/<key>` and with their source line, and re-runs re-map only what changed; vanished claims are retracted, `--prune` handles vanished records. `--watch` tails a file; `--query` answers over store plus source without writing. A `source/<name> HAS path:` claim declares a source in-band: `cave connect` with no argument runs them, and a text store follows them on open.

CHAPTER 15

# Letting a Model Read

Most knowledge arrives as prose: a call summary, a supplier's email, a page on a website. `cave ingest` hands that text to a language model and stores what the model writes, with provenance, digests, and an atomic commit. This chapter explains the contract with the model, what a good extraction looks like, and how to run the pipeline without a model when you want to see the machinery.

## 15.1 The agent contract

`cave ingest` does not contain a language model and does not depend on any SDK. It runs a shell command you supply, the **agent**, once per batch of files, with the extraction prompt on standard input. The agent either records claims itself through the MCP server that `cave ingest` starts for it, or, with `--stdout`, prints CAVE text that `cave ingest` lints and stores. Any headless agent works:

```
$ cave ingest --db roastery.db 'notes/**/*.md' \
    --agent 'claude -p --mcp-config {mcp-config} --allowedTools "mcp__cave__*"'

$ cave ingest --db roastery.db 'notes/**/*.md' --stdout \
    --agent 'copilot -p "$(cat {prompt-file})"'
```

The placeholders `{prompt-file}`, `{mcp-config}`, and `{db}` are substituted by the engine and shell-quoted, so write them bare. The command runs under `/bin/sh` on POSIX and PowerShell on Windows; its output streams are bounded, and a timeout terminates the whole process tree.

## 15.2 What the prompt asks for

The prompt carries the extraction rules from spec §14 and a summary of what the store already knows, so the model reuses established names instead of inventing variants. `--dry-run` prints the plan and the first prompt without running anything:

notes.md

```
Call with Ana at La Cima, 12 August.

The Huila lot is being re-priced to 8.20 USD/kg from September; the
contract is indexed, so expect 8.60 by next year. Ana thinks the
Tolima lot will cup around 85 but the sample is not in yet. La Cima is
now certified organic (paperwork attached).
```

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ cave ingest --db roastery.db notes.md --stdout --dry-run --agent cat | sed -n '1,3p;/^##
Existing/,/^Established/p'
ingest plan: 1 source(s) in 1 batch(es), 0 skipped (unchanged), 0 rejected
  batch 1: notes.md

## Existing knowledge

The database currently holds 33 current belief(s).
Established entities (use these names, do not invent variants): lot/yirgacheffe-26 (6), lot/
huila-26 (6), lot/santa-ana-26 (6), coffee/morning-blend (5), la-cima (4), kaffa-coop (3),
lot (3), coffee/yirgacheffe (3), cafe/north (3), SUPPLIES (2), verb (2), STOCKS (2),
supplier (2), coffee (2), cafe (2)
```

The rules the model is asked to follow are the ones a careful human follows too: one claim per line; resolve pronouns to entities; record decisions rather than the discussion around them; keep code and exact strings in backticks; drop filler; preserve uncertainty with `@ N%` and `+/-`; cite the smallest supporting line range; and never extract credentials or data that must be selectively erased later, because the store is permanent. `--instructions <file>` adds your own domain guidance, such as which verbs and entity names to use. `--embed` inlines file contents into the prompt, numbered by line, for agents that cannot read files themselves.

Here is what a good extraction of the notes looks like. It is the file we will use as a stand-in for the model:

`extracted.cave`

```
lot/huila-26 HAS price: 8.20 USD/kg @src:notes.md#L3 @2026-09..
lot/huila-26 HAS price: 8.20 -> 8.60 USD/kg @src:notes.md#L3-L4 @2026-09..2027-09 @ 70% ;
"expect 8.60 by next year"
lot/tolima-26 HAS score: ~85 @src:notes.md#L4-L5 @ 50% ; sample not in yet
la-cima HAS certification: organic @src:notes.md#L5-L6
```

Each claim cites its lines, the guess about Tolima carries the hedge the speaker gave it, and the trajectory records the indexed contract as a range rather than as two unrelated numbers.

## 15.3 Running the pipeline without a model

Because the agent is just a command, `cat` of a prepared file is a valid agent. That makes every other part of the pipeline observable:

```
$ cave ingest --db roastery.db notes.md --stdout --agent 'cat extracted.cave'
ingest (strict): 1 source(s) matched, 0 skipped (unchanged), 1 batch(es), applied
batch 1/1 (1 file(s)): +4 claim(s)
source accepted: notes.md — batch 1
done: +4 claim(s)

$ cave query --db roastery.db 'la-cima HAS certification: ?c'
?c = organic

$ cave ingest --db roastery.db notes.md --stdout --agent 'cat extracted.cave'
ingest (strict): 1 source(s) matched, 1 skipped (unchanged), 0 batch(es), applied
source skipped: notes.md
done: +0 claim(s)
```

The second run reads nothing, because each source is remembered by content digest in a bookkeeping claim; edit the file and only it is re-ingested. Claims that name no source are stamped `@src:ingest`, a stable actor across re-runs, so a re-extracted fact supersedes in its series instead of forking a new one.

Ingestion is atomic by default: an exact SQLite snapshot preserves the current claims and provenance in an isolated stage. If a fetch, the agent, or the lint fails, nothing reaches the database. The final merge also rejects changes to existing identities; success means the staged run was committed. `--lenient` commits accepted batches, continues past failures, exits 1 if anything was rejected, and reports every source (`--json` for the manifest). Rejected sources keep no digest and retry next time. Sources may be paths, globs, or URLs; web pages are fetched and reduced to their readable text before the model sees them.

## 15.4 Operating modes for a model

When you talk to a model directly rather than through `cave ingest`, three modes keep the conversation clean. In **extract mode** ("cave this") the model emits only CAVE. In **query mode** ("what do we know about la-cima?") it translates the question into a pattern. In **normal mode** it goes back to prose. The portable CAVE Agent Skill in the repository packages this guidance for Copilot, Codex, Claude Code, and other hosts (Chapter 23).

**Where the tokens go**
Prose goes to the model; structured records do not. A spreadsheet through `cave ingest` would cost tokens to produce claims that `cave connect` produces exactly and for free. Use the model for the part only a reader can do.

> **In short.** `cave ingest <sources> --agent '<command>'` runs any headless agent per batch with the prompt on stdin, through MCP or `--stdout`. The prompt carries the extraction rules and the established names. Sources are digested and skipped when unchanged; runs are atomic unless `--lenient`. A `cat` of a prepared file is a valid agent, which makes the pipeline testable.

CHAPTER 16

# Measuring an Extraction

Is the new prompt better than the old one? Does this model name things consistently? Questions like these stay anecdotal until there is a number. `cave eval` turns a source, its expected extraction, and a few questions into a score that any agent can be run against, as many times as you like.

## 16.1 A fixture

A case is a golden file with its source beside it. The golden is the extraction you would accept; an optional queries file lists patterns the built store must answer, however the agent chose to name things:

evals/roastery.md

```
Call with Ana at La Cima, 12 August.

The Huila lot is being re-priced to 8.20 USD/kg from September; the
contract is indexed, so expect 8.60 by next year. Ana thinks the
Tolima lot will cup around 85 but the sample is not in yet. La Cima is
now certified organic (paperwork attached).
```

evals/roastery.golden.cave

```
lot/huila-26 HAS price: 8.20 USD/kg @2026-09..
lot/tolima-26 HAS score: ~85 @ 50%
la-cima HAS certification: organic
```

evals/roastery.queries.cave

```
; what the built store must answer, written as cave query prints it

la-cima HAS certification: ?c
  ?c = organic

; the hedged score must not pass a confidence filter

lot/tolima-26 HAS score: ?s
  WHERE conf >= 0.8
  none
```

## 16.2 Scoring

The agent extracts each source into a fresh throwaway store, and the result is scored against the golden by claim key and value. Actor stamps are ignored, so it does not matter whether the agent wrote through MCP or stdout, and a claim written in the inverse direction matches for free. A `cat` of the golden is the trivially perfect agent, which is a good way to check that a fixture is consistent with itself:

```
$ cave eval evals --stdout --agent 'cat roastery.golden.cave'
eval: 1 case(s), 1 run(s) each
evals/roastery: 3 golden claim(s), 2 query(ies), source roastery.md
  run 1/1: 3 claim(s) — 3 matched; P 100% R 100% F1 100%; queries 2/2
suite: P 100% R 100% F1 100%; queries 100%
```

Now simulate the naming drift real extractions suffer, with an agent that capitalizes the supplier:

```
$ cave eval evals --stdout --agent 'sed "s/la-cima/La-Cima/" roastery.golden.cave'
eval: 1 case(s), 1 run(s) each
evals/roastery: 3 golden claim(s), 2 query(ies), source roastery.md
  run 1/1: 3 claim(s) — 2 matched; P 67% R 67% F1 67%; queries 1/2
    miss: la-cima HAS certification: organic
    extra: La-Cima HAS certification: organic
    query failed: la-cima HAS certification: ?c
      missing ?c = organic
suite: P 67% R 67% F1 67%; queries 50%
```

Precision, recall, and F1 are reported per run and for the suite, with the misses, extras, and failed bindings diagnosed so the cause is visible. The agent command runs with the case directory as its working directory, which is why `cat roastery.golden.cave` works.

## 16.3 Making it a gate

`--runs 3` runs every case three times to measure variance. `--min 90%` exits 1 unless the mean F1 and the query pass rate reach the threshold, which is all it takes to put an extraction prompt under continuous integration. `--tolerance 5%` accepts numeric values within a relative band. `--judge '<command>'` runs a second agent over the leftovers after strict scoring, pairing claims that are semantically equivalent despite naming drift into a separate **judged** F1 that never replaces the strict one. Point a real agent at the suite exactly as you would at `cave ingest`:

```
$ cave eval evals --runs 3 --min 90% \
    --agent 'claude -p --mcp-config {mcp-config} --allowedTools "mcp__cave__*"'
```

## 16.4 Scoring a reconstruction

The same harness scores memory **reconstruction** (Chapter 20). A case whose stem has a `.loop.cave` sibling is a reconstruction eval: the source is the knowledge, the loop file declares where to start and how far to walk, and the golden is what a good walk should collect.

`loop/flat-taste.cave`

```
flat-taste EXISTS @cafe/harbour @ 90%
stale-lot CAUSE flat-taste @ 50%
grinder/harbour CAUSE flat-taste @ 30%
lot/huila-26 IS stale-lot @ 60%
burr-swap FIX flat-taste @ 40%
cafe/harbour HAS manager: "Sam"
cafe/harbour STOCKS coffee/morning-blend
```

`loop/flat-taste.golden.cave`

```
stale-lot CAUSE flat-taste @ 50%
grinder/harbour CAUSE flat-taste @ 30%
lot/huila-26 IS stale-lot @ 60%
burr-swap FIX flat-taste @ 40%
```

`loop/flat-taste.loop.cave`

```
loop SEEDS flat-taste
loop HAS query: `why does the morning blend taste flat at the harbour cafe?`
loop HAS steps: 3
```

Without an agent the deterministic heuristic policy runs, which is the baseline every model-driven policy is measured against:

```
$ cave eval loop
eval: 1 case(s), 1 run(s) each
loop/flat-taste: 4 golden claim(s), reconstruction over flat-taste.cave
  run 1/1: 5 claim(s) — 4 matched; P 80% R 100% F1 89%
    extra: flat-taste EXISTS @cafe/harbour @ 90%
suite: P 80% R 100% F1 89%
```

With `--agent`, the model picks each expansion instead, and "does the model beat the heuristic" becomes two runs of the same command.

> **In short.** A case is `<stem>.golden.cave` beside its source, plus optional `<stem>.queries.cave` expectations. `cave eval <suite> --agent '<command>'` scores claim-key F1 and query pass rate per run; `--runs`, `--tolerance`, `--judge`, and `--min` make it a CI gate. A `.loop.cave` sibling scores a reconstruction instead, with the heuristic as baseline.

PART

# Concluding and acting

Rules that conclude what nobody wrote, actions that record decisions under conditions, automations that react to new claims, and reconstruction that walks the graph from a symptom.

CHAPTER 17

# Rules

Everything so far stores, checks, or asks. Nothing concludes. A rule takes patterns that already match and records a claim nobody wrote, with the reasons attached. This chapter covers the rule line, how confidence flows through it, what a derivation leaves behind, and why re-running it is safe.

## 17.1 The rule line

A rule is a comma-separated conjunction of **premises**, an arrow, and one **conclusion**. Premises are CAVE-Q patterns or constraints; the conclusion is an ordinary claim whose slots may be variables. `=>` is the only new token.

rules.cave

```
; what follows from what we already know

DEPENDS-ON IS verb ; coffee X depends on supplier Y
DEPENDS-ON REVERSE SUPPLIES-FOR

?c USES ?lot, ?s SUPPLIES ?lot => ?c DEPENDS-ON ?s ; a coffee depends on whoever supplies
its lots
?lot HAS score: ?s, ?s >= 86 => ?lot IS specialty ; 86 and up is specialty grade
?lot HAS stock: ?kg, ?kg < 10 kg => ?lot NEEDS reorder ; below ten kilos we reorder
```

Lines with `=>` are rules; everything else in the file is prelude that is validated and ingested first. An invalid prelude appends nothing and declares no rules, even on a cached retry; correct it before retrying. Declaration errors make `cave derive` exit nonzero, though already-stored rules still derive. Premises evaluate left to right over current, positive, non-retracted beliefs, each binding narrowing the next pattern, so inverse verbs and transitive hops cost nothing extra. A constraint (`?s >= 86`) tests a variable an earlier premise bound; numeric comparison applies when both sides are numbers, and a unit on the constraint demands the same unit on the value. Every variable in the conclusion must be bound by a premise. `NOT` in a premise matches explicitly negated claims; there is no negation-as-failure, because “no claim says otherwise” is not evidence.

## 17.2 Firing

Load the store and the stock levels from Chapter 14, then derive:

stock.csv

```
lot,kg,roasted
lot/yirgacheffe-26,42,2026-08-20
lot/huila-26,6,2026-08-25
lot/santa-ana-26,15,2026-08-22
```

stock.map.cave

```
; one row per lot in the green store

?lot HAS stock: ?kg kg
?lot HAS last-roasted: ?roasted
```

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)
```

```
$ cave connect stock.csv --map stock.map.cave --db roastery.db --key lot
connect: 3 record(s): 3 mapped, 0 skipped (unchanged); +6 claim(s)

$ cave derive --db roastery.db rules.cave
declared 3 rule(s), +2 prelude claim(s)
rule/ab991fceb047: 3 solution(s), +2 appended, 0 updated, 0 retracted, 0 unchanged ; a
coffee depends on whoever supplies its lots
rule/8c447ade9060: 1 solution(s), +1 appended, 0 updated, 0 retracted, 0 unchanged ; 86 and
up is specialty grade
rule/89c4640b638e: 1 solution(s), +1 appended, 0 updated, 0 retracted, 0 unchanged ; below
ten kilos we reorder
derived: +4 appended, 0 updated, 0 retracted, 0 unchanged (2 pass(es))

$ cave query --db roastery.db '?c DEPENDS-ON ?s'
?c = coffee/morning-blend  ?s = la-cima
?c = coffee/yirgacheffe  ?s = kaffa-coop

$ cave query --db roastery.db '?lot NEEDS reorder'
?lot = lot/huila-26
```

The first rule found three solutions but appended two claims: the morning blend uses two lots from the same supplier, and two solutions concluding the same claim key produce one conclusion. Rules fire to a fixpoint, so a conclusion can feed another rule in the next pass.

## 17.3 Rules are claims, and conclusions carry their reasons

A rule is stored in-band as an attribute claim under `rule/<digest>`, where the digest is the first twelve hex characters of a hash over the rule's normalized text; whitespace variants share a digest and re-declaring an unchanged rule appends nothing. `cave derive --list` shows them, and `cave derive` with no file fires whatever the store already holds.

Every derived claim is stamped `@src:rule/<digest>`, and the export shows why it is believed: `BECAUSE` edges to the exact premise rows and a `VIA` edge to the rule:

```
$ cave export --db roastery.db | grep -A2 '^lot/huila-26 NEEDS reorder'
lot/huila-26 NEEDS reorder @src:rule/89c4640b638e
  BECAUSE lot/huila-26 HAS stock: 6 kg @src:connect/stock/lot-huila-26 @src:stock.csv#L3
  VIA rule/89c4640b638e HAS rule: `?lot HAS stock: ?kg, ?kg < 10 kg => ?lot NEEDS reorder`
@src:cave-derive ; below ten kilos we reorder
```

The chain runs all the way to a line in a CSV. Derived confidence is the product of the premise confidences and an optional factor on the conclusion (`... => ?x NEEDS ?y @ 80%`), the noisy-AND of Chapter 5 under an explicit independence assumption. When several solutions conclude the same key, the strongest derivation wins, never the sum, so many weak paths cannot outvote one good one. Conclusions below a floor (five percent by default) are not asserted.

A rule's output is one belief series per conclusion, separate from any hand-written series about the same fact, and rule conclusions sit at the bottom of the resolution ladder (Chapter 11): a derived claim never outranks a human's.

## 17.4 Re-running is safe

Run it again and nothing happens:

```
$ cave derive --db roastery.db
rule/ab991fceb047: unchanged premises, skipped ; a coffee depends on whoever supplies its
lots
rule/8c447ade9060: unchanged premises, skipped ; 86 and up is specialty grade
rule/89c4640b638e: unchanged premises, skipped ; below ten kilos we reorder
derived: +0 appended, 0 updated, 0 retracted, 0 unchanged (1 pass(es))
```

Two mechanisms make that true. A conclusion equal to its current belief is skipped, so a loop never accretes identical rows. And each rule records a **watermark**, the highest transaction it accounted for; a later run re-fires a rule when some newer row could change one of its premises, judged by shape and deliberately ignoring mutable values, tags and confidence, so a replacement or retraction re-fires the rules its claim used to feed. A companion vocabulary fingerprint also wakes rules when qualifier edges change the active declarations without adding claim rows. It also captures alias matching and the confidence floor: changing either re-evaluates support. Old watermarks without that complete fingerprint are re-evaluated once. Vocabulary changes during a run restart support calculation within the pass budget. `--full` ignores watermarks.

Derivation reserves its write transaction before selecting rules or reading watermarks and refreshes vocabulary inside it. Another writer's committed rule revocations and inverse declarations therefore apply even when the derivation connection was already open.

Support is recomputed on every firing. An ordinary conclusion the rule no longer reaches is retracted at zero confidence, and while a rule re-establishes its derivations they are invisible to premise matching, so a retracted premise retracts the dependent chain across rules and two derivations cannot keep each other alive. Restock the Huila lot and the reorder disappears:

`stock.csv`

```
lot,kg,roasted
lot/yirgacheffe-26,42,2026-08-20
lot/huila-26,36,2026-08-25
lot/santa-ana-26,15,2026-08-22
```

```
$ cave connect stock.csv --map stock.map.cave --db roastery.db --key lot
connect: 3 record(s): 1 mapped, 2 skipped (unchanged); +2 claim(s)
  note: prelude unchanged, skipped

$ cave derive --db roastery.db
rule/ab991fceb047: unchanged premises, skipped ; a coffee depends on whoever supplies its
lots
rule/8c447ade9060: unchanged premises, skipped ; 86 and up is specialty grade
rule/89c4640b638e: 0 solution(s), +0 appended, 0 updated, 1 retracted, 0 unchanged ; below
ten kilos we reorder
derived: +0 appended, 0 updated, 1 retracted, 0 unchanged (2 pass(es))

$ cave query --db roastery.db '?lot NEEDS reorder'
no matches
```

`cave derive --retract <rule>` retracts a rule and its ordinary conclusions. Vocabulary declarations (`IS verb`, `REVERSE`, `RENAMED-TO`) remain additive: losing their supporting premise or retracting the producing rule does not remove them from the registry. Retraction selects all current declaration contexts and checks digest-prefix ambiguity inside its write transaction; an ambiguous prefix changes nothing. `--dry-run` reports inside a rolled-back transaction; `--aliases` lets premises match through the alias closure. The MCP `cave_derive` tool has the same semantics, so an agent can declare rules with an ordinary append and fire them without leaving the protocol.

> **In short.** `premise, premise, ?v op value => conclusion ; label` is a rule. Premises match current positive beliefs left to right; conclusions are stamped `@src:rule/<digest>` with `BECAUSE` edges to premise rows and a `VIA` edge to the rule. Confidence multiplies; the strongest derivation wins. Re-runs are idempotent and watermark-incremental, and lost support retracts conclusions.

CHAPTER 18

# Actions

Rules fire on their own. A decision should not. An **action** is a named write that a caller invokes with parameters, that checks its preconditions against current belief, and that appends its effects atomically or not at all. It is how a human at the terminal or an agent over MCP records a decision without a free-form append.

## 18.1 Declaring an action

An action is declared like a rule, as one attribute claim whose value is the body, under a stable name:

actions.cave

```
; governed writes: the only way an order gets recorded

action/reorder HAS action: `?lot, ?lot NEEDS reorder, ?lot SUPPLIED-BY ?supplier => ?lot HAS
order: 30kg, ?supplier NEEDS contact` ; order 30 kg of a lot that ran low
action/reorder/lot IS param ; the lot to reorder
```

The body is the rule line with three additions. A bare variable on the left (`?lot`) is a **parameter** the caller supplies. The right side may list several effects. And an effect's `@ N%` is the confidence it is asserted with, not a product of premise confidences: an action is the caller's assertion, and its premises are gates, not evidence. The companion claim `action/reorder/lot IS param` documents the parameter; its comment surfaces in `cave act --list` and in the MCP tool schema.

The declaration call validates the whole prelude before appending it or declaring actions. A prelude error leaves the call without writes; corrected retries work normally, and successful unchanged preludes append nothing.

The name is the identity. Redeclaring appends to the same claim key, so an action has exactly one current definition and its evolution is an ordinary belief series. Retracting the declaration disables the action; effects of past executions were true when recorded and stay.

## 18.2 Executing

Set up the store from Chapter 17, with the Huila lot low and the reorder rule fired:

stock.csv

```
lot,kg,roasted
lot/yirgacheffe-26,42,2026-08-20
lot/huila-26,6,2026-08-25
lot/santa-ana-26,15,2026-08-22
```

stock.map.cave

```
?lot HAS stock: ?kg kg
```

rules.cave

```
?lot HAS stock: ?kg, ?kg < 10 kg => ?lot NEEDS reorder ; below ten kilos we reorder
```

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)
```

```
$ cave connect stock.csv --map stock.map.cave --db roastery.db --key lot
connect: 3 record(s): 3 mapped, 0 skipped (unchanged); +3 claim(s)

$ cave derive --db roastery.db rules.cave
declared 1 rule(s)
rule/89c4640b638e: 1 solution(s), +1 appended, 0 updated, 0 retracted, 0 unchanged ; below
ten kilos we reorder
derived: +1 appended, 0 updated, 0 retracted, 0 unchanged (2 pass(es))

$ cave act --db roastery.db --declare actions.cave
declared 1 action(s), +1 prelude claim(s)
```

Now execute it, twice, and then try it on a lot that does not need reordering:

```
$ cave act --db roastery.db reorder lot=lot/huila-26
executed action/reorder: +2 appended, 0 updated, 0 unchanged (1 solution(s))
  appended: lot/huila-26 HAS order: 30kg
  appended: la-cima NEEDS contact

$ cave act --db roastery.db reorder lot=lot/huila-26
executed action/reorder: +0 appended, 0 updated, 2 unchanged (1 solution(s))
  unchanged: lot/huila-26 HAS order: 30kg
  unchanged: la-cima NEEDS contact

$ cave act --db roastery.db reorder lot=lot/santa-ana-26
cave act: precondition failed — no current belief satisfies "?lot NEEDS reorder"
[exit 1]
```

Execution follows a fixed sequence. The current declaration is resolved and parsed. Arguments are validated: every parameter supplied, no unknown names, values formatted exactly as connect formats record fields. Premises evaluate left to right with the parameters pre-bound; a premise with no solution fails the action and nothing is appended, which is what the third call shows. Variables used in effects must bind uniquely across the surviving solutions; an ambiguous binding fails too, because an action executes once, deterministically, or not at all. Then the effects append in one transaction. The second call appended nothing because every effect already equalled its current belief.

The write transaction starts before the declaration, premises, or baseline shape violations are read, and refreshes the stored vocabulary first. A competing writer cannot revoke a prerequisite between its validation and the effect commit; a revocation committed before the write lock is acquired is seen by the action's checks.

## 18.3 What an execution leaves behind

Effects are stamped `@src:action/<name>` and carry lineage: `BECAUSE` edges to the premise rows of the justifying solution and a `VIA` edge to the declaration. The reasoning nests all the way down:

```
$ cave export --db roastery.db | grep -A6 '^lot/huila-26 HAS order'
lot/huila-26 HAS order: 30kg @src:action/reorder
  BECAUSE
    lot/huila-26 NEEDS reorder @src:rule/89c4640b638e
      BECAUSE lot/huila-26 HAS stock: 6 kg @src:connect/stock/lot-huila-26 @src:stock.csv#L3
      VIA rule/89c4640b638e HAS rule: `?lot HAS stock: ?kg, ?kg < 10 kg => ?lot NEEDS
reorder` @src:cave-derive ; below ten kilos we reorder
    la-cima SUPPLIES lot/huila-26 @src:cli
  VIA action/reorder HAS action: `?lot, ?lot NEEDS reorder, ?lot SUPPLIED-BY ?supplier => ?
lot HAS order: 30kg, ?supplier NEEDS contact` @src:cave-act ; order 30 kg of a lot that ran
low
```

An order, because the lot needed reordering, because its stock was six kilos on line 3 of a CSV, through a named rule; and because la-cima supplies it, through a named action. Nobody wrote a sentence of that.

## 18.4 The gate, and hooks

Actions run inside the shape gate of Chapter 13 by default: if an execution would introduce a new `EXPECTS` violation, it rolls back. `--no-check` opts out, and `--dry-run` reports inside a rolled-back transaction without firing anything.

For library callers, an enclosing store transaction is still allowed when the action will not fire a configured hook. Otherwise the action rejects and rolls back its own savepoint: the enclosing transaction could still be cancelled after a hook escaped. Run that action outside the caller-owned transaction or omit hook configuration. No-op actions and dry runs remain nestable; hook delivery is never silently deferred.

A decision recorded in the store often needs to reach the outside world. Executable content must never live in the store, so the action **names** a hook and the command lives in configuration:

`hooks.json`

```
{ "purchase": "cat >> purchase-orders.log" }
```

```
$ echo 'action/reorder HAS hook: purchase' | cave add --db roastery.db
added 1 claim(s), 0 edge(s)

$ printf 'lot/santa-ana-26 HAS stock: 4 kg\n' | cave add --db roastery.db
added 1 claim(s), 0 edge(s)

$ cave derive --db roastery.db | tail -n 1
derived: +1 appended, 0 updated, 0 retracted, 1 unchanged (2 pass(es))

$ cave act --db roastery.db reorder lot=lot/santa-ana-26 --hooks hooks.json
executed action/reorder: +1 appended, 0 updated, 1 unchanged (1 solution(s))
  appended: lot/santa-ana-26 HAS order: 30kg
  unchanged: la-cima NEEDS contact
hook purchase: ok

$ cat purchase-orders.log
lot/santa-ana-26 HAS order: 30kg
```

The hook runs strictly **after** the commit, with the appended claims as CAVE text on standard input and `{action}` and `{param}` placeholders substituted shell-quoted. A failing hook cannot un-happen recorded knowledge; it is reported with the claims intact. A hook that is named but not configured is reported as not fired, which makes running without hook configuration a legitimate side-effect-free mode. Idempotent no-op executions and dry runs never fire hooks, so a loop never re-notifies the world about claims that did not change. Hook configuration comes from `--hooks`, or `$CAVE_HOOKS`.

## 18.5 Serving actions to agents

`cave mcp` generates one tool per current action, `act_<name>`, with the declaration comment as description and the parameters as a schema (Chapter 23). Agents then get a vocabulary of allowed writes with validated preconditions, atomic appends, and provenance, instead of free-form `cave_add`, and the MCP client's ordinary permission prompt is where a human confirms.

> **In short.** action/ HAS action: `?param, premise => effect, effect` declares a governed write. Execution validates parameters, requires every premise to match current belief, needs unique bindings, appends effects atomically with `@src:action/<name>` and lineage, is idempotent, and runs inside the shape gate. `HAS hook: <name>` names an out-of-band command that runs after commit. Actions become `act_<name>` MCP tools.

CHAPTER 19

# Automations

Rules fire when you run `cave derive`, actions when you call them, and connect re-maps when a file changes. Nothing yet watches the store. An **automation** pairs a trigger pattern with steps and fires when new claims match; `cave automate` is the loop that evaluates them. With connect feeding one end, sense, model, conclude, act, and record close on one machine, unattended.

## 19.1 Declaring an automation

An automation is declared like a rule and named like an action:

automations.cave

```
; whenever a lot runs low, order more

automation/reorder-low-stock HAS automation: `?lot NEEDS reorder => action/reorder` ; low
stock triggers a reorder
```

The left side is the trigger: ordinary premises, at least one of them a pattern, and no bare parameters, because an automation has no caller and every binding must come from the trigger. The right side is a list of steps, each one of:

- `action/<name>`: execute the action, its parameters bound from trigger variables of the same name;
- `hook/<name>`: run the named hook from the same configuration actions use, with the triggering rows on standard input;
- a quoted prompt: send it to an agent, bound variables substituted once, and record the agent's CAVE reply.

Prompt substitution reads only template tokens. Inserted values that contain another variable name or replacement text such as `$&` remain literal.

Like rule text and action bodies, the declaration is pure data. It names things to run; the commands themselves stay in configuration.

As with rules and actions, the declaration call validates the whole prelude before using its digest cache. A prelude error appends nothing and declares no automations; corrected retries work normally without duplicating partial rows.

## 19.2 Arming, and the first cycle

Rebuild the store from Chapter 18 and declare the automation:

stock.csv

```
lot,kg,roasted
lot/yirgacheffe-26,42,2026-08-20
lot/huila-26,6,2026-08-25
lot/santa-ana-26,15,2026-08-22
```

stock.map.cave

```
?lot HAS stock: ?kg kg
```

rules.cave

```
?lot HAS stock: ?kg, ?kg < 10 kg => ?lot NEEDS reorder ; below ten kilos we reorder
```

actions.cave

```
action/reorder HAS action: `?lot, ?lot NEEDS reorder, ?lot SUPPLIED-BY ?supplier => ?lot HAS
order: 30kg, ?supplier NEEDS contact` ; order 30 kg of a lot that ran low
```

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ cave connect stock.csv --map stock.map.cave --db roastery.db --key lot
connect: 3 record(s): 3 mapped, 0 skipped (unchanged); +3 claim(s)

$ cave derive --db roastery.db rules.cave | tail -n 1
derived: +1 appended, 0 updated, 0 retracted, 0 unchanged (2 pass(es))

$ cave act --db roastery.db --declare actions.cave
declared 1 action(s)

$ cave automate --db roastery.db --declare automations.cave
declared 1 automation(s)

$ cave automate --db roastery.db --once
settled: 0 firing(s) over 1 pass(es); derived +0 appended, 0 updated, 0 retracted
```

The Huila lot already needs a reorder, and the first cycle is quiet. That is by design: an automation arms at the moment it is declared, and rows recorded before that are **state**, not **events**. A solution fires only when it cites at least one row newer than the automation's watermark.

## 19.3 Something happens

The Santa Ana lot runs low. Overwrite the stock file, connect it, and run one cycle:

stock.csv

```
lot,kg,roasted
lot/yirgacheffe-26,42,2026-08-20
lot/huila-26,6,2026-08-25
lot/santa-ana-26,4,2026-08-27
```

```
$ cave connect stock.csv --map stock.map.cave --db roastery.db --key lot
connect: 3 record(s): 1 mapped, 2 skipped (unchanged); +1 claim(s)

$ cave automate --db roastery.db --once
automation/reorder-low-stock: fired 1 solution(s) ; low stock triggers a reorder
  ?lot = lot/santa-ana-26
    action/reorder: ok (+2 appended, 0 updated, 0 unchanged)
settled: 1 firing(s) over 2 pass(es); derived +1 appended, 0 updated, 0 retracted

$ cave query --db roastery.db '?lot HAS order: ?kg'
?lot = lot/santa-ana-26  ?kg = 30kg
```

Read the cycle from the inside. Each pass first fires the store's rules, incrementally, so the new stock row produced a new `NEEDS reorder` conclusion. Then every current automation is evaluated; the conclusion is newer than the watermark, so the solution fired, and the action ran with `?lot` bound from the trigger, under all of its own checks. The next pass saw the action's effects, found nothing new to fire, and the cycle settled. The Huila lot, low since before the declaration, was correctly left alone.

## 19.4 Why the loop cannot run away

Three rules keep the loop honest. The watermark is appended **before** any step runs, so a crash between the two loses that batch's outside-world steps rather than replaying them; a re-run must never re-notify the world. An automation is deaf to its own echo: rows stamped with its own agent replies or with the

effects of its own action steps are never events for it, while another automation's output triggers normally, which is how automations chain. And every write path is idempotent, so a cycle converges unless something genuinely new keeps arriving; a pair of automations whose agents keep answering each other with fresh values is a design error that the pass guard bounds per cycle but cannot prevent.

Each batch reserves a write transaction, refreshes its declaration and vocabulary, evaluates triggers, and commits the watermark before running steps. Two runners cannot claim the same batch. A declaration revoked while an earlier agent runs is checked again before its batch is claimed; already-claimed steps continue. Agent execution holds no write lock. The `settle` API rejects caller-owned transactions because a savepoint cannot commit the firing log before external execution.

The report's `complete` flag requires a quiet final pass and completed rule derivation. Hitting the pass limit prints `incomplete` and makes `--once` exit nonzero; retrying keeps committed watermarks, so steps are not replayed. If rule derivation reaches its separate limit, run `cave derive` with a higher `--max-passes` before retrying. Malformed stored rules also make `--once` fail when derivation is enabled; they appear in the report's `problems` list while valid declarations continue. `--no-derive` skips rule evaluation and those checks.

Programmatic callers can pass `signal` to `settle` or `watchCycle`. Cancellation stops further claims and steps and discards late agent replies, while committed watermarks stay committed; unfinished steps in a claimed batch are not replayed. Custom completion callbacks must cancel their own work. The daemon yields between repeated settles so shutdown callbacks can run even on a busy store.

Declaring is arming, and re-declaring after a retraction does not replay the rows recorded while the automation was off. Retractions fire nothing, unchanged re-assertions append no row and so fire nothing, and a value update fires because the new row is the current one.

## 19.5 Running it

`cave automate` without `--once` is the daemon: one settle cycle at startup, then a cheap poll of the store's highest transaction every two seconds, and a cycle whenever it moves. `--once` is one cycle and an exit code that carries step failures, which is all a cron job needs. `--hooks` and `--agent` supply the hook configuration and the agent command for prompt steps, with the same shell contract as `cave ingest`:

```
$ cave automate --db roastery.db --hooks hooks.json --agent 'claude -p'
watching (poll every 2s, ctrl-c to stop)
```

The loop is deliberately not an MCP tool: it is a process the operator runs. The **declarations** are ordinary claims, though, so an agent can declare an automation through an ordinary append and a running loop serves it from the next cycle.

> **In short.** automation/ HAS automation: `trigger => action/x, hook/y, "prompt"` fires its steps for each solution that cites a row newer than its watermark; rows older than the declaration are state, not events. Each cycle fires rules, evaluates automations, and repeats until nothing fires. Watermark first, no self-echo, idempotent writes. `--once` for cron, no flag for a daemon.

CHAPTER 20

# Reconstruction

A query answers the question you knew to ask. Sometimes you only have a symptom. `cave reconstruct` starts from an entity, walks the graph outward along forward and reverse edges, and collects related claims until a budget runs out. It is the **agent layer**: a policy over the graph, not part of the language, and deliberately kept outside the specification.

## 20.1 A symptom

Customers at the harbour cafe say the morning blend tastes flat. Record what is known and suspected:

`tasting.cave`

```
; the July cupping: what we tasted, and what we think explains it

flat-taste EXISTS @cafe/harbour @ 90% ; two complaints in a week
stale-lot CAUSE flat-taste @ 50%
grinder/harbour CAUSE flat-taste @ 30%
water/harbour CAUSE flat-taste @ 20%
lot/huila-26 IS stale-lot @ 60% ; roasted a month ago
lot/huila-26 HAS roast-date: 2026-07-30
grinder/harbour HAS burr-age: 14mo
burr-swap FIX flat-taste @ 40%
cafe/harbour HAS manager: "Sam"
```

```
$ cave add --db roastery.db roastery.cave tasting.cave
added 42 claim(s), 0 edge(s)

$ cave reconstruct --db roastery.db flat-taste --steps 4 --trace
; 1. flat-taste @ 1.00 +5 claim(s)
; 2. stale-lot @ 0.40 +1 claim(s)
; 3. burr-swap @ 0.32 +0 claim(s)
; 4. grinder/harbour @ 0.24 +1 claim(s)

flat-taste EXISTS @cafe/harbour @src:cli @ 90% ; two complaints in a week
stale-lot CAUSE flat-taste @src:cli @ 50%
grinder/harbour CAUSE flat-taste @src:cli @ 30%
water/harbour CAUSE flat-taste @src:cli @ 20%
burr-swap FIX flat-taste @src:cli @ 40%
lot/huila-26 IS stale-lot @src:cli @ 60% ; roasted a month ago
grinder/harbour HAS burr-age: 14mo @src:cli
```

The trace lines are comments, so the output is valid CAVE that can be piped into another store. Starting from the symptom, the walk collected the three hypotheses and the proposed fix, then expanded the strongest hypothesis and found the stale lot, then the grinder and its burr age. With four steps it did not reach the cafe's manager or the lot's price, which is the point: reconstruction is about what is **related**, ranked, not everything reachable.

## 20.2 The policy

At each step the loop has a frontier of entities it could expand next, and it scores them by relation direction, confidence, recency, and novelty. The deterministic heuristic picks the highest score. `--steps` bounds the number of expansions, `--claims` stops after enough claims are collected, and `--trace` shows every choice with its score.

With `--agent`, a language model makes the select-or-stop decision instead. Each step sends one prompt containing the question, the claims collected so far as canonical CAVE, and the scored frontier; the model replies with the name of the cue to expand or `STOP`. Scoring stays local arithmetic, and an unparseable reply degrades to the strongest cue rather than ending the walk:

```
$ cave reconstruct --db roastery.db flat-taste \
    --query 'why does the morning blend taste flat at the harbour cafe?' \
    --agent 'claude -p'
```

The heuristic is not a fallback; it is the baseline. Reconstruction fixtures in `cave eval` (Chapter 16) score both policies with the same claim-key F1, so "the model retrieves better memory" is a claim you can test with two commands.

## 20.3 Why not a vector search

Reconstruction follows explicit, typed relations, in both directions, through the reverse names the vocabulary declared. It returns the source claims themselves, not summaries, and it exposes the path that brought each claim into context. A similarity search over embeddings finds text that sounds like the question; a walk over a claim graph finds what the store believes is connected to it, with the confidence of every hop visible.

The same loop is served to agents as the MCP `cave_reconstruct` tool, and `cave demo` narrates a small multi-hop recovery on an in-memory store.

> **In short.** `cave reconstruct <seed>` walks forward and reverse edges best-first from a cue, collecting related claims as canonical CAVE text; `--trace` shows the path and scores, `--steps` and `--claims` bound it. The deterministic heuristic is the baseline; `--agent` lets a model choose each expansion, and `cave eval` scores both the same way.

PART

# Sharing what the store knows

Documents that cite their claims, a browser page over the store, agents that read and write through MCP, and stores that merge without conflicts.

CHAPTER 21

# Documents That Cite Their Claims

A query answers you. A document is for someone else, and it should say where every fact came from. `cave report` renders a Markdown template against the store, splicing values into prose and repeating fragments per match, and footnotes query-produced facts with the claims behind them. Ordinary template prose passes through unchanged; it is not automatically cited or checked against the store.

## 21.1 A template

A template is ordinary Markdown with two live constructs. An inline splice, `cave-q: pattern` , drops the single value a sentence needs. A fenced `cave-q` block holds a pattern, optional `WHERE` lines, and a fragment that is rendered once per solution with the variables substituted:

brief.md

````
# Buying brief

The Huila lot is priced at `cave-q: lot/huila-26 HAS price: ?p` and
scored `cave-q: lot/huila-26 HAS score: ?s`.

## Lots by supplier

```cave-q
?s SUPPLIES ?lot
- **?lot** from ?s [^?]
```

## Specialty grade

```cave-q
?lot HAS score: ?score
WHERE value >= 86
- ?lot scored ?score
```
````

Everything else passes through verbatim: headings, prose, tables, hand-written footnotes, fenced code in any other language. The template is a document, not knowledge; it lives in a file under version control, like a mapping or a hook configuration, never in the store.

## 21.2 Rendering

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ cave report --db roastery.db brief.md
# Buying brief

The Huila lot is priced at 7.80 USD/kg[^c1] and
scored 84[^c2].

## Lots by supplier

- **lot/yirgacheffe-26** from kaffa-coop [^c3]
- **lot/huila-26** from la-cima [^c4]
```

```
- **lot/santa-ana-26** from la-cima [^c5]

## Specialty grade

- lot/yirgacheffe-26 scored 87 [^c6]

[^c1]: `lot/huila-26 HAS price: 7.80 USD/kg @src:cli` — <date>, claim key `["e:lot/
huila-26","HAS",0,"a:price",["src:cli"]]`
[^c2]: `lot/huila-26 HAS score: 84 @src:cupping/june` — <date>, claim key `["e:lot/
huila-26","HAS",0,"a:score",["src:cupping/june"]]`
[^c3]: `kaffa-coop SUPPLIES lot/yirgacheffe-26 @src:cli` — <date>, claim key `["e:kaffa-
coop","SUPPLIES",0,"r:e:lot/yirgacheffe-26",["src:cli"]]`
[^c4]: `la-cima SUPPLIES lot/huila-26 @src:cli` — <date>, claim key `["e:la-
cima","SUPPLIES",0,"r:e:lot/huila-26",["src:cli"]]`
[^c5]: `la-cima SUPPLIES lot/santa-ana-26 @src:cli` — <date>, claim key `["e:la-
cima","SUPPLIES",0,"r:e:lot/santa-ana-26",["src:cli"]]`
[^c6]: `lot/yirgacheffe-26 HAS score: 87 @src:cupping/june` — <date>, claim key `["e:lot/
yirgacheffe-26","HAS",0,"a:score",["src:cupping/june"]]`
```

Every rendered solution cites its stored row. The marker lands at the fragment's `[^?]` placeholder when there is one and is appended to the last line otherwise; inline splices always append. The definitions collect at the end of the document: the row's canonical line, exactly as `cave export` prints it with actor stamps included, the date it was recorded, and the claim key, so a reader can pull the full history behind any sentence. When a claim carries a source span, the footnote adds the decoded location, linked when the source is a URL. Labels are `c1, c2, …` in order of first citation, skipping labels already used in the original template, ignoring case and including code examples, so hand-written footnotes retain their meaning.

Only template occurrences are substituted. Text inside a stored binding that looks like another variable or a `[^?]` placeholder remains unchanged.

A fragment shaped like a bullet renders a list; one shaped like a table row renders rows under a hand-written header; a paragraph with a trailing blank line renders prose. A block with no fragment renders each solution as `cave query` prints it, as a cited bullet. A query with no solutions renders nothing, which is the honest shape of an empty section.

## 21.3 Splices are deterministic or nothing

An inline splice must bind exactly one variable and match exactly one solution. When a second source scores the Huila lot, the sentence cannot be rendered honestly:

```
$ echo 'lot/huila-26 HAS score: 86.5 @src:q-grader/ana @ 80%' | cave add --db roastery.db
added 1 claim(s), 0 edge(s)

$ cave report --db roastery.db brief.md 2>&1 | sed -n '3,4p;/^template line/p'
The Huila lot is priced at 7.80 USD/kg[^c1] and
scored *(ambiguous: 2 matches)*.
template line 4: ambiguous inline splice "lot/huila-26 HAS score: ?s": 2 matches — several
series contest the fact; --resolve picks the §26 winner
[exit 1]

$ cave report --db roastery.db brief.md --resolve | sed -n '3,4p'
The Huila lot is priced at 7.80 USD/kg[^c1] and
scored 84[^c2].
```

The document still renders, with the problem marked in place and reported on standard error with the template line number, and the command exits 1. A contested fact is ambiguity working as intended; `--resolve` is the knob, and the footnote then shows exactly which source the sentence stands on.

## 21.4 The same report, another day

Every query in a template takes the same read options as `cave query`. `--as-of` renders the report as belief stood at a past moment; `--at` anchors it in valid time so one template renders "the plan as of mid-year" or "next year's projection"; `--aliases` widens names; `--max-sensitivity` raises the audience ceiling above the default `internal`. The template stays under version control while the store evolves, and the document is re-rendered from current belief on demand. Reports are deliberately not an MCP tool: an agent composing prose already reads through the query tools, and the report is the human's reproducible deliverable.

> **In short.** A template is Markdown plus `cave-q: pattern` splices and fenced `cave-q` blocks with a per-solution fragment. `cave report` renders it and footnotes query-produced facts with the canonical claim, date, and claim key. Splices must be unambiguous; `--resolve` picks a winner. `--as-of`, `--at`, `--aliases`, and `--max-sensitivity` apply to every query in the document.

CHAPTER 22

# Looking at the Store

Everything so far serves programs. `cave serve` is for the person: one self-contained web page over the store, strictly read-only, bound to localhost, with no build step and no external resource. This chapter walks through what it shows and what it promises.

## 22.1 Starting it

```
$ cave serve --db roastery.db
serving roastery.db at http://127.0.0.1:2283/ (sensitivity <= internal, read-only, ctrl-c to
stop)
```

The default port is 2283, which spells "cave" on a phone keypad; `--port 0` picks a free one and `--host` widens the bind deliberately. The page is a single HTML document with inline style and script. It loads nothing from anywhere, so it works offline and under a strict content security policy: the browser can render the store but cannot call out. Claims are rendered from the stored columns and side tables, never by re-parsing text, so there is no second grammar to drift out of sync.

## 22.2 The views

**Dashboard.** The health report of Chapter 13 on a screen: a one-line summary with the coverage tiles a click away, a map of the most connected entities, topics, then the frontier, which is shape violations, review candidates, stale beliefs, and alias disagreements, each folded under its count, and the latest appends. It answers "what is missing and what needs a look" from the graph itself.

**Entity 360.** One name's current picture: its types, its attributes and metrics, its relations in both directions with the declared inverse name annotated on the object side, its topics, and the alias closure on an explicit toggle, because alias-aware reading is opt-in everywhere. A neighborhood graph draws the name and its neighbors one to three hops out, and a **related** index lists those neighbors grouped by relation, so the next entity is one click away. Underneath, folded, the activity feed of the newest rows about the name, superseded and retracted included.

**Belief history.** One claim key's series, oldest first, as a timeline with confidence bars. The last row is current belief; retraction and supersession are visible instead of destroyed.

**Lineage.** The edge table walked both ways from one row: **cites** (its `BECAUSE` premises, `VIA` rules and actions, `WHEN` conditions: why this is believed) and **cited by** (what depends on it). Edges form a graph and the render is a tree, so a row reached again is restated without children; the walk is depth-capped and a truncated node says so rather than posing as a leaf.

**Graph.** Current relations between entities as a node-link diagram, laid out in the page itself: the whole store's most connected names, or one entity's neighborhood full screen. Edges are stored rows, never inferred; negated relations are dashed, uncertain ones faded and labelled, and a capped graph says how many names it left out. Every node is a link.

**Search.** The store's full-text index over subjects, objects, values, comments, and raw lines, newest first, with matching entity names offered first as direct links.

Every entity name, claim key, and row id links onward, so the whole store is reachable by clicking. A claim row shows its statement and confidence; the contexts, tags, comment, timestamp, and the history and lineage links open from the row. Pressing `/` jumps to search. Source spans are decoded and, for URLs, linked to the cited fragment.

## 22.3 The promises

**Read-only, structurally.** Only `GET` and `HEAD` are answered and no endpoint writes. Recording knowledge stays with `cave add`, the MCP tools, and the kinetic layer. Every request reads the live store, so a running automation's appends show on the next refresh. Responses are marked `no-store`; `HEAD` preserves the status and cache protection of `GET` without a body. The page ignores responses from earlier navigation, so a slow request cannot replace the selected view. A malformed URL fragment shows an error and leaves normal navigation available.

**Local by default.** The store is one person's knowledge on one machine. There is no authentication layer and none is planned; wider serving belongs behind your own transport, and what it would share is the selected sensitivity view, read-only.

**Sensitivity-scoped.** The ceiling defaults to `internal` and is raised only with `--max-sensitivity`. Counts, aliases, history, lineage, and search are computed from visible rows only; an edge is served only when both endpoints are visible.

## 22.4 Scripting it

The page reads plain JSON endpoints, which `curl` can read too: `/api/overview`, `/api/entity?name=`, `/api/topic?name=`, `/api/history?key=`, `/api/lineage?id=`, `/api/search?q=`, `/api/graph` (optionally `?name=` and `&depth=` from 1 to 3), and `/api/names?q=`, with `&aliases=1` where the closure applies. They serve exactly the views above; CAVE-Q remains the query language. The same view models are plain functions in the `@cavelang/view` package, usable without a server.

```
$ cave add --db roastery.db roastery.cave
$ cave serve --db roastery.db --port 0 > serve.log 2>&1 &
$ pid=$!
$ tries=50
$ until grep -q 'http://' serve.log; do \
    kill -0 "$pid" 2>/dev/null && [ $((tries -= 1)) -gt 0 ] || break; sleep 0.2; done
$ url=$(grep -o 'http://[^ ]*/' serve.log | head -1)
$ [ -n "$url" ] || { cat serve.log; kill "$pid" 2>/dev/null; false; }
$ curl -s "${url}api/entity?name=lot/huila-26" | jq '.types'
$ kill "$pid"; wait "$pid" 2>/dev/null || true
```

The server prints its address only once it is listening, so the loop waits for the log line, and stops waiting when the server process has exited or ten seconds have passed. The guard that follows then fails the step: it prints the log, stops the server if it is still alive, and returns a nonzero status, so a store that cannot be opened shows its error and a script under `set -e` stops there instead of calling `curl` with no host. The last line stops the server again (`wait` reports the killed server's signal status, which `|| true` discards so that `set -e` does not mistake it for a failure): `cave serve` runs until told otherwise, and every `--port 0` start picks a fresh port, so a forgotten server is an orphan holding the database open. In a script, `trap 'kill "$pid"' EXIT` right after the start does the same on every exit path.

The view is deliberately not an MCP tool. Agents read through the query and neighbourhood tools; the page is for the human outside the loop.

> **In short.** `cave serve` renders a dashboard, relation graphs, entity pages, belief-history timelines, lineage trees, and search from one self-contained page. `GET`/`HEAD` only, localhost, `internal` ceiling by default, live reads. JSON endpoints under `/api/` serve the same views.

CHAPTER 23

# Agents Over MCP

A language model with tools is the most demanding user a store has: it writes quickly, names things inconsistently, and forgets what it recorded an hour ago. `cave mcp` serves the store to any Model Context Protocol client and gives the model a small, self-describing vocabulary: read, search, walk, reconstruct, fuse, derive, and, when an operator allows it, record and act.

## 23.1 The server

`cave mcp` speaks the protocol over standard input and output, which is how MCP clients start servers. Registering it with a client is one line:

```
$ copilot mcp add cave -- cave mcp --db "$HOME/cave.db"
$ claude mcp add cave -- cave mcp --db "$HOME/cave.db"
```

The server is self-describing. Its initialization instructions carry the compact syntax card, and the `cave_help` tool serves version-matched guidance on demand (`overview`, `write`, `find`, `revise`, `safety`), so a model that has never seen CAVE can look up how to use it before it writes. A portable CAVE Agent Skill in the repository adds workflow guidance for Copilot, Codex, Claude Code, and other skill hosts; it defers to the connected server's help for exact syntax.

## 23.2 The tools

| Tool | What it does |
|---|---|
| `cave_help` | Version-matched operating guidance; reads no user data. |
| `cave_query` | A bounded CAVE-Q page with `all`, `aliases`, `asOf`, `at`, `resolve`, `limit`, `cursor`. |
| `cave_about` | Current claims about one entity, both directions. |
| `cave_neighbors` | Named forward and inverse edges, for walking the graph by hand. |
| `cave_search` | Full-text search over claims, values, tags, and comments; `raw`, `limit`. |
| `cave_reconstruct` | The reconstruction loop from seed cues (Chapter 20). |
| `cave_fuse` | Precision-weighted fusion of numeric estimates of one quantity (Chapter 5). |
| `cave_lint` | Validate CAVE text without storing it. |
| `cave_add` | Append CAVE text; lenient by default, `strict` on request. |
| `cave_derive` | Fire the store's rules (Chapter 17). |
| `cave_export` | Sensitivity-scoped canonical text. |
| `act_<name>` | One generated tool per current action declaration (Chapter 18). |

Claim results are canonical CAVE text, not JSON rows: that is the agent-facing compatibility contract, and a model reads a claim line more reliably than a nested object. Because the protocol is newline-delimited JSON-RPC, the server can be driven from a shell with no client at all, which is a good way to see exactly what a model sees:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ printf '%s\n' \
    '{"jsonrpc":"2.0","id":1,"method":"initialize","params":
```

```
{"protocolVersion":"2025-06-18","capabilities":{},"clientInfo":
{"name":"book","version":"0"}}}' \
    '{"jsonrpc":"2.0","method":"notifications/initialized"}' \
    '{"jsonrpc":"2.0","id":2,"method":"tools/list"}' \
    '{"jsonrpc":"2.0","id":3,"method":"tools/call","params":
{"name":"cave_query","arguments":{"pattern":"?who SUPPLIES lot/huila-26"}}}' \
    | cave mcp --db roastery.db 2>/dev/null > replies.jsonl

$ grep -o '"name":"cave_[a-z_]*"' replies.jsonl | tr '\n' ' '
"name":"cave_help" "name":"cave_add" "name":"cave_query" "name":"cave_fuse"
"name":"cave_search" "name":"cave_about" "name":"cave_neighbors" "name":"cave_reconstruct"
"name":"cave_derive" "name":"cave_export" "name":"cave_lint"

$ grep -o '"text":"[^"]*"' replies.jsonl
"text":"?who = la-cima  ; la-cima SUPPLIES lot/huila-26"
```

One JSON reply per request lands in the file: the handshake, the tool list with every tool's schema, and the query result as CAVE text. The server exits when its input closes. Tool failures come back as `isError` results so the model can correct a call, and the startup banner goes to standard error because standard output is protocol only.

## 23.3 Provenance for agents

Claims a model appends without a `@src:` context are stamped `@src:agent/<client-name>`, using the name the client gave at initialization. That puts every agent in its own belief series and in the agent precedence class, below a human at the terminal (Chapter 11). `--src pipeline/nightly` replaces the stamp for a named pipeline; `--no-src` disables it.

## 23.4 Scoping what an agent may do

The surface has four permission classes: `read` retrieves stored data, `evaluate` performs ephemeral computation, `record` appends durable data, and `action` may execute governed effects. `--permissions` serves only the named classes, `--tools` only the named tools, and `--read-only` is the shorthand that keeps read and evaluate. Scopes compose by intersection, and a scope that names an unknown tool, or serves nothing, fails at startup before the database is opened. The one thing startup cannot check is an `act_<name>` tool: those exist only while the action is declared, and are read from the store on every listing, so the third scope below serves an empty list until Chapter 18's declaration has been loaded, and a misspelt action name is an empty list rather than an error. The declaration is Chapter 18's:

`actions.cave`

```
; governed writes: the only way an order gets recorded

action/reorder HAS action: `?lot, ?lot NEEDS reorder, ?lot SUPPLIED-BY ?supplier => ?lot HAS
order: 30kg, ?supplier NEEDS contact` ; order 30 kg of a lot that ran low
action/reorder/lot IS param ; the lot to reorder
```

```
$ cave mcp --db roastery.db --read-only
$ cave mcp --db roastery.db --permissions read,evaluate
$ cave act --db roastery.db --declare actions.cave
$ cave mcp --db roastery.db --permissions action --tools act_reorder
$ cave mcp --db roastery.db --tools cave_query,cave_about,cave_search
```

Tools outside the scope are absent from the tool list and indistinguishable from nonexistent when called. Read and evaluate tools carry the protocol's read-only hint so clients can auto-approve them, and the client's ordinary permission prompt is where a human confirms a write.

## 23.5 Actions as the write vocabulary

The most useful scope for a working agent is often no `cave_add` at all. Every current action declaration becomes an `act_<name>` tool, recomputed on every tool listing, with the declaration comment as its description and the parameters as its schema. The agent then chooses from a set of allowed decisions whose preconditions the engine checks, whose effects append atomically with lineage, and whose hooks reach the outside world only after commit (`--hooks` supplies the configuration). The store gets governed writes instead of free-form ones, and the operator can widen the vocabulary by declaring another action, without restarting anything.

## 23.6 Driving an agent from the command line

The same contract runs the other way. `cave ingest --agent`, `cave eval --agent`, `cave reconstruct --agent`, `cave suggest-alias --agent`, and `cave automate --agent` each run a headless agent command with a prompt on standard input and, where the agent records through the store, a temporary MCP configuration substituted for `{mcp-config}`. Any agent that can be started from a shell fits, and the model never enters the engine: there is no SDK dependency, the shell is explicit, output is bounded, and a timeout kills the whole process tree.

**What an agent should do**

Recall before reasoning: ask `cave_about` for one entity, `cave_query` for a shape, `cave_search` when the wording is unknown, `cave_reconstruct` for bounded multi-hop context. Record one fact per line with a source and honest confidence, lint unfamiliar syntax first, never edit history, and never store secrets. That paragraph is the Agent Skill in miniature.

> **In short.** `cave mcp --db <store>` serves the store over stdio with read, search, walk, reconstruct, fuse, derive, lint, add, export, and generated `act_<name>` tools. Appends are stamped `@src:agent/<client>`. `--read-only`, `--permissions`, and `--tools` scope the surface by intersection. The reverse contract, `--agent '<command>'`, lets every CAVE workflow drive any headless agent.

CHAPTER 24

# Two Stores Become One

Knowledge accumulates on more than one machine: a laptop and the shop counter, a store and its air-gapped copy. Eventually one must absorb the other. The data model has already done the hard part, because contradictory claims coexist and resolve at read time, so valid replicas merge without belief conflicts. This chapter covers `cave sync`, the annotated text form that carries row identity, and the git workflow they compose into.

## 24.1 Rows have global identity

Every appended row is minted one UUIDv7 that serves as its id and its transaction. Replay requires canonical lowercase UUIDv7 ids; an invalid batch changes neither rows nor the receive clock. Merging copies rows that are absent **by id**, verbatim, and skips matching rows the target already has. A reused id with different claim data or explicit provenance rejects the whole source before adding rows, lineage, or a merge record. Different raw spelling and metadata order are compatible, as is provenance inferred during migration or copying. Dry runs perform the same checks, and the command exits nonzero on a rejected source. Build two stores that share a history and then diverge:

```
$ cave add --db laptop.db roastery.cave
added 33 claim(s), 0 edge(s)

$ cave backup --db laptop.db --out shop.db >/dev/null

$ echo 'cafe/harbour HAS manager: "Sam"' | cave add --db shop.db
added 1 claim(s), 0 edge(s)

$ echo 'lot/huila-26 HAS score: 86.5 @src:q-grader/ana @ 80%' | cave add --db laptop.db
added 1 claim(s), 0 edge(s)

$ cave sync --db laptop.db shop.db
merged 1 claim(s), 0 edge(s), 33 already present
record: store/shop SYNCED-INTO store/laptop ; +1 claim(s), +0 edge(s)

$ cave sync --db laptop.db shop.db
merged 0 claim(s), 0 edge(s), 34 already present
```

Everything follows from identity preservation. Re-running merges nothing. Merges are transitive: after the shop absorbs a third store, syncing the shop into the laptop carries the third store's rows under their original ids, and a later direct sync finds them present. Two stores syncing each other converge. And the same fact recorded independently on both machines arrives as two rows in one belief series, asserted twice, which is what happened; resolution and fusion apply at read time exactly as within one store. Rows are never re-stamped on merge, because re-minting ids would fork identity and duplicate every row on the next sync.

## 24.2 The merge is a claim

A merge that changed anything is recorded in the target store as an ordinary claim, `store/<from> SYNCED-INTO store/<into>`, stamped `@src:sync`, with the batch counts in its comment. The labels default to file basenames and can be set with `--as` and `--into`; the claim key is one per origin-and-target pair, so its belief series is the sync log. A sync that merged nothing appends nothing. `--dry-run` computes the full report inside a rolled-back transaction.

The `syncDb` API owns its transaction: it rejects calls inside a caller-owned transaction before attaching or copying, even for dry runs. SQLite cannot detach a source used by an uncommitted outer transaction. Use

a standalone database sync, or `syncText` with a complete annotated export when the merge must follow the caller's commit or rollback.

## 24.3 The receive rule

A store that absorbs rows from a fast-clocked machine holds rows stamped ahead of its own wall clock. If its next append sorted before them, it would silently lose currency to the merged past. So every append receives a transaction greater than every transaction already in the store: the id generator observes the maximum on open and after every merge and never mints below it. Local appends always win locally. Merged history interleaves by origin clocks, so cross-machine recency is physical time, not causality; where trust should outrank recency, that is precedence (Chapter 11), not transaction order.

## 24.4 Text that carries identity

Canonical export deliberately omits transaction ids, which is right for restoring and wrong for merging. `--tx` adds them as a full-line comment above each claim, and `cave sync` accepts that text, or `-` for standard input:

```
$ cave export --db shop.db --tx --max-sensitivity restricted | grep -B1 'manager'
  ;@ <uuid>
  HAS manager: "Sam" @src:cli

$ cave export --db shop.db --tx --max-sensitivity restricted --out shop.cave
exported 34 claim(s) to shop.cave

$ cave sync --db laptop.db shop.cave --as shop-export
merged 0 claim(s), 0 edge(s), 34 already present
```

The annotation sits at the claim's own indentation, here inside the factored `cafe/harbour` block. Every other reader sees ordinary comments, so `cave import` of an annotated file is an ordinary replay with fresh ids, and every existing consumer reads the file unchanged. `cave sync` is strict the other way: every claim line must be annotated with a well-formed id, or the file is rejected whole, because a half-annotated file would merge half a store idempotently and duplicate the rest on every run. A shared row cited by several parents is restated with the same id and unions back into one row on replay; the same id with different content forks identity and rejects the file. Text sync also checks IDs already stored in the target before adding rows or lineage. Canonical interchange content must agree; raw spelling and metadata ordering can differ without changing that content. `--max-sensitivity restricted` makes the export complete; a `--current` export with `--tx` is a seed whose rows keep their identity, so a store grown from it merges back without duplication.

Text sync reserves its write transaction before refreshing vocabulary and validating or canonicalizing the text. Vocabulary committed by another writer before the reservation therefore applies during replay, even on an already-open connection. A dry run restores the prior registry and UUID generator state as well as rolling back rows.

**Provenance in annotated text**
When contexts cannot reconstruct stored provenance, the transaction annotation adds a JSON payload with four sets: actors, sources, runs, and domains. Replay preserves them exactly, including empty sets, and rejects malformed or conflicting metadata before writing. Bare legacy annotations still work; older sync readers must upgrade to read annotations with payloads.

## 24.5 The store under git

Annotated text can store the exported claims, lineage, and provenance. Commit `knowledge.cave`, never the store file, regenerated before every commit and never edited by hand. A branch is a git branch plus a private store file, rebuilt from the text as plumbing. Take the laptop as the live store and open a branch that adds a supplier:

suppliers.cave

```
finca-elena IS supplier ; met at the Bogotá fair
finca-elena SUPPLIES lot/tolima-26
```

```
$ cave export --db laptop.db --tx --max-sensitivity restricted --out knowledge.cave
exported 37 claim(s) to knowledge.cave

$ git init -q && git switch -c reorg-suppliers
Switched to a new branch 'reorg-suppliers'

$ cave sync --db work.db knowledge.cave --no-record
merged 37 claim(s), 0 edge(s)

$ cave add --db work.db suppliers.cave
added 2 claim(s), 0 edge(s)

$ cave export --db work.db --tx --max-sensitivity restricted --out knowledge.cave
exported 39 claim(s) to knowledge.cave
```

The pull-request diff is the appended claims: review new `;@` ids as the semantic additions, and treat a changed physical line around an existing id as presentation, since canonical output may factor an old and a new row through a shared prefix. Valid replicas have no belief conflicts; a **text** merge can, when two branches append at the end of the file. Never hand-merge the export. Rebuild it as the union the two texts already are. Suppose the live store moved on while the branch was open, so its export is the other side of the conflict:

```
$ echo 'cafe/north HAS manager: "Kim"' | cave add --db laptop.db
added 1 claim(s), 0 edge(s)

$ cave export --db laptop.db --tx --max-sensitivity restricted --out theirs.cave
exported 38 claim(s) to theirs.cave

$ cave sync --db merge.db knowledge.cave --no-record
merged 39 claim(s), 0 edge(s)

$ cave sync --db merge.db theirs.cave --no-record
merged 1 claim(s), 0 edge(s), 37 already present

$ cave export --db merge.db --tx --max-sensitivity restricted --out knowledge.cave
exported 40 claim(s) to knowledge.cave
```

Git can run that as a merge driver for `*.cave` files, with the branch's text and the other side's as the two inputs; the ancestor is unused because union by identity needs no three-way merge. Landing the branch is one more sync into the live store, and this one is a real merge event, so let it record. Refreshing a stale branch is the same move pointed the other way, as a checkout.

Costs, stated plainly: a branch is a full copy of the store, with no shared structure, and divergence between the committed text and anyone's live store is bounded by the next sync, not prevented. Both are the right trade at CAVE's scale, one machine and one store file, with plain text as the escape hatch. Sync is deliberately not an MCP tool: store files are machine-local paths, and distribution is the operator's concern.

> **In short.** `cave sync --db target <source>` merges rows by UUIDv7 identity from a store file or `;@`-annotated text; present rows skip, re-runs merge nothing, and an effective merge appends a `SYNCED-INTO` record. Every append outsorts everything merged. `cave export --tx --max-sensitivity restricted` carries claims and lineage you can commit; rebuild working stores from it with `--no-record`, and resolve text conflicts by re-exporting the union.

PART

# Under the hood

How claims are stored and normalized, how the packages fit together, the optional formal-reasoning layer, and advice for running a store for real.

CHAPTER 25

# How Claims Are Stored

In the current implementation a store is one SQLite file with a small relational schema; the schema is a detail behind the `cave` commands and the package APIs, not part of the language. This chapter describes the tables, the pipeline that turns a line of text into rows, the one rule that makes reverse readings free, and how the schema itself is versioned.

## 25.1 The tables

Claims occupy one central table. Contexts, provenance, tags, and claim-to-claim edges live in side tables, and a full-text index covers the human-readable columns:

```
cave_claim(id, tx, subject, verb, negated,
           object, attribute, value_text, value_num, value_unit, value_approx,
           delta_text, delta_num, delta_unit, sigma_level,
           conf, importance, comment, raw_line, claim_key)

cave_context(claim_id, context)
cave_provenance(claim_id, dimension, value)     -- actor, source, run, domain
cave_tag(claim_id, key, value)                  -- value NULL for a flat tag
cave_edge(parent_id, role, child_id)            -- WHEN, VIA, BECAUSE, QUALIFIES
cave_fts(claim_id, subject, verb, object, attribute, value_text, comment, raw_line)
```

`id` and `tx` are both the row's UUIDv7, which encodes its wall-clock millisecond and sorts chronologically. `subject`, `verb`, and `object` are stored in the canonical primary direction; `raw_line` keeps the text exactly as written, inverse spelling and all. `value_num` and `value_unit` hold the parsed number and normalized unit when the value is numeric, with the original in `value_text`; a trajectory keeps `value_num` empty. An edge's role is the qualifier that produced it, with two normalizations: an `UNLESS` line is stored as a `WHEN` edge to a negated child, and a full claim grouped under another line links to it with `QUALIFIES`. Indexes cover the claim key with transaction, subject, verb, object, attribute, confidence, each context, each tag key and value, and each edge end.

## 25.2 From a line to rows

Before storage, every line goes through one canonicalization pipeline:

1. Uppercase the verb.
2. Resolve a renamed spelling to its stable storage verb; then, if the verb is a declared inverse, swap subject and object and substitute the primary verb.
3. Resolve continuation lines by filling the inherited endpoint from the parent.
4. Normalize entity whitespace to hyphens; keep proper-noun casing.
5. Preserve `raw_line` exactly.
6. Parse confidence to a decimal (`@ 90%` becomes `0.9`).
7. Parse the approximate marker into `value_approx`.
8. Normalize multipliers (`20B` becomes `20000000000`).
9. Keep both the raw value text and the parsed number.
10. Split tags into key and value.
11. Stamp actor provenance if the line names no source (Chapter 8).
12. Compute the claim key on the canonical form.
13. Write the row and its side-table rows in one transaction.

The ordering is the invariant the rest of the system rests on: **canonicalize before identity**. An inverse spelling and a continuation must have converged on the primary form before the key is computed, or one fact would fork into several series.

Ingestion reserves its transaction before canonicalizing. Registry access and inverse reads also refresh vocabulary after peer commits, including on read-only connections. Unchanged data reuses the cache; rollback restores it and its data-version marker.

## 25.3 Inverses are views

The store holds one row per fact. A forward read uses the subject index; a reverse read uses the object index and names the relation through the inverse registry built from `REVERSE` claims:

```
-- forward: what does X supply?
SELECT object FROM cave_claim WHERE subject = 'la-cima' AND verb = 'SUPPLIES';

-- reverse: who supplies Y? (named SUPPLIED-BY by inverse_of('SUPPLIES'))
SELECT subject FROM cave_claim WHERE object = 'lot/huila-26' AND verb = 'SUPPLIES';
```

Materializing the reverse direction would double every row, split each belief series in two, and double the work of contradiction resolution. Keeping inverses lazy costs one index that exists anyway.

## 25.4 Current belief in SQL

The query that everything else builds on is a join from each claim key to its newest transaction. As-of reads restrict the inner query to a transaction boundary; resolution ranks the rows it returns within their groups; the alias closure is a recursive common-table expression over the symmetrized `ALIAS` edges:

```
SELECT c.* FROM cave_claim c
JOIN (SELECT claim_key, MAX(tx) AS max_tx FROM cave_claim
      WHERE tx <= :boundary GROUP BY claim_key) latest
  ON c.claim_key = latest.claim_key AND c.tx = latest.max_tx;
```

## 25.5 Schema versions

Every store records an integer schema version in `PRAGMA user_version`. Version 0 is the unversioned legacy format and version 2 is the current schema. Version 2 adds an index led by transaction id, speeding head reads and single-claim appends without rewriting stored knowledge. Opening a store reads the version before anything else. An older supported store gets each forward migration in ascending order, and each migration's DDL, data backfill, structural validation, and version advance share one immediate transaction, so an interruption leaves either the old version or the complete next one. A store newer than the runtime fails immediately by name and is never opened on a guess; database sync rejects such a source for the same reason. A current-version store is validated for its required tables, indexes, and columns rather than silently repaired.

Migrations are forward-only. Before an upgrade that needs a rollback point, stop every process using the store, close it, and copy the closed file (or take a `cave backup`); rolling back means restoring that copy under the same stopped-writer discipline with a compatible runtime.

> **Two portable forms**
> Canonical text is the interchange format: readable, diffable, replayable with fresh ids. The exact snapshot from `cave backup` is the temporal backup: every id, transaction, and side-table row. Neither is proprietary; both are readable without CAVE.

> **In short.** One `cave_claim` table in canonical direction plus context, provenance, tag, edge, and full-text side tables. A single pipeline uppercases, resolves renames and inverses, expands continuations, parses values, stamps provenance,

> and only then computes the claim key. Reverse reads are index views, never rows. `user_version` gates forward-only transactional migrations.

CHAPTER 26

# How the Pieces Fit

CAVE is a pnpm workspace of TypeScript packages that form a ladder from domain types to user surfaces. This chapter is the map: what each layer owns, which way dependencies point, how the same kernel reaches a terminal, an agent, a browser, and an editor, and the handful of invariants every change has to preserve.

## 26.1 Three boundaries

Three decisions shape everything else.

**Text becomes data once.** CAVE text is parsed and canonicalized before it is keyed or persisted; reads operate on stored columns and side tables and never re-parse `raw_line`. There is one parser, one key rule, one query engine, and one persistence model, reused from every surface.

**Belief changes append.** An update or retraction is a new row in the same belief series. Existing rows stay addressable for history, provenance, bitemporal queries, and synchronization.

**Policy is mostly knowledge.** Verb declarations, rules, actions, automations, shape expectations, source reliability, and precedence are stored as claims. The executable machinery stays in code; the configuration it interprets travels with the knowledge, and executable commands (hooks, agents) never enter the store.

## 26.2 The layers

| Layer | Packages | Responsibility |
|---|---|---|
| Domain | `core`, `fusion` | Immutable claim and value types, keys, time, UUIDv7, probabilistic math. |
| Language | `parser`, `canonical` | CAVE text and diagnostics, the inverse and lifecycle registry, canonical claims, emission. |
| Data | `store`, `query`, `shape` | Persistence, CAVE-Q, resolution, expectations, health, write gates. |
| Formal reasoning | `solver`, `scenario`, optional `solver-z3` | Exact portable models, typed snapshot bindings, opt-in Z3 search (Chapter 27). |
| Behavior | `rules`, `act`, `automate`, `loop` | Derivation, governed writes, event processing, reconstruction policies. |
| Movement | `connect`, `ingest`, `sync` | Deterministic records, agent extraction, store union. |
| Integration | `mcp`, `eval` | The agent tool protocol and repeatable quality evaluation. |
| Presentation | `cli`, `view` | Command dispatch, the read-only HTTP view, cited reports. |
| Language tooling | `tree-sitter-cave`, `highlight`, the VS Code extension | One grammar and highlight query for terminal and editors. |
| Browser | `website` | Documentation and a playground running the kernel in the browser on an in-memory store. |

**Taking work out of the playground**
The in-memory database belongs to its browser tab. **Download claims** saves the current editor text as a `.cave` file, including unapplied edits. Keep the claims you want to save in the editor: appended claims absent from that text are not included in the download. **Copy result** saves the displayed output to the clipboard with its submitted query and valid-time context. If clipboard access fails, **Select result text** selects that output for manual copying.

Dependencies point inward, toward domain, language, and data. Higher packages compose lower functions; the lower layers never call the CLI, MCP, HTTP, or agent surfaces. The code favours small functions and immutable values over class hierarchies, with conventional error subclasses carrying typed failure details at API boundaries.

Release preparation cleans and rebuilds outputs from its own version checkout. Tests and smoke precede publishing or tagging. Missing packages publish only for the current main version; older complete releases can recover tags. Registry errors halt publication, and absence requires an unambiguous npm `E404` code.

Workspace boundaries and release boundaries differ on purpose. Libraries with independent consumers publish as their own npm packages. Rules, actions, automation, ingestion, MCP, views, and the other command implementations stay as focused private workspace packages and ship as documented `@cavelang/cli/<feature>` subpaths, so tests and ownership stay small without multiplying version surfaces. Every public package releases at one version, in lockstep, through automated changesets.

## 26.3 One process boundary

Every command enters one awaited dispatcher, whether its implementation is synchronous, asynchronous, or a long-running server. `SIGINT` and `SIGTERM` become one abort signal; HTTP servers, protocol readers, watchers, timers, and stores close before the conventional signal exit code is returned. A simple user error is one stack-free line on standard error; a command that reports findings, such as parse diagnostics or a rejected shape gate, prints its structured report instead (Chapter 28). `CAVE_DEBUG=1` keeps the diagnostic stack for unexpected errors.

Local integrations cross one process boundary too. Direct commands are executable-and-arguments arrays with no shell interpolation. Agent and hook strings are explicitly platform-shell templates, `/bin/sh` on POSIX and PowerShell 7 on Windows, with placeholder values quoted for that shell. The boundary bounds both output streams separately, normalizes exits, redacts command material from failure messages, and owns whole-tree termination on timeout, cancellation, or overflow.

## 26.4 One grammar, four renderers

A tree-sitter grammar lives beside the parser and is the source for every presentation of CAVE text: `cave highlight` and the colours `cave export` prints on a terminal, the website's editor, the VS Code extension's semantic tokens, and any tree-sitter-native editor pointed at the grammar. The grammar and parser are tested against each other; one query drives every renderer. VS Code frees each document tree after use and its parser and query on disposal or failed activation.

## 26.5 The invariants

Changes to the system are expected to preserve these:

1. Canonicalize before identity.
2. Never update or delete belief rows.
3. Treat row ids as global identities that sync preserves with their transaction order, side tables, raw text, keys, and edges.
4. Filter publication structurally, from visible rows only; a current-only export resolves current belief before applying the ceiling.
5. Format source spans in one place.

6. Keep generated clients derived and reproducible.
7. Keep provenance dimensions separate from identity-bearing contexts.
8. Version every physical schema change with one transactional migration.
9. Publish snapshots only after verification.
10. Keep reads non-destructive: aliasing, resolution, valid time, and reconstruction never rewrite claims.
11. Use the store's transaction boundary for compound writes.
12. Keep external effects after commit and out of the store.
13. Reuse the kernel from every surface.

## 26.6 Boundaries that will stay

Some things are deliberately not built. Ordinary stored claims stay fully bound: variables live only in queries, rules, actions, automations, and connector templates. Claims are not reified into terms; explicit qualifier edges and provenance rows address relationships instead. Time-dependent values are linear trajectories and tiled steps, not stored formulas. Network listeners stay outside the core: a webhook or socket bridge owns its transport and feeds a watched file or a bounded connect pass. There is no multi-tenant access control and none is planned; the store is one person's or one team's knowledge on one machine, with plain text as the escape hatch. A proposal to move one of these boundaries has to show a workflow the existing structures cannot express and specify identity, scope, determinism, security, lifecycle, and compatibility before the boundary moves.

> **In short.** Packages form a ladder from domain types to surfaces, with dependencies pointing inward and one kernel reused everywhere. Text becomes data once, belief appends, policy is knowledge. One dispatcher and one process boundary own lifecycle and shell safety; one grammar drives every renderer. Thirteen invariants and a short list of permanent non-goals keep the shape stable.

CHAPTER 27

# Formal Reasoning

CAVE-Q retrieves beliefs and rules derive claims, but neither searches a space of possible assignments. The optional formal-reasoning layer handles feasibility, optimization, counterexamples, and bounded sensitivity over inputs taken from the store, without turning a hypothetical model into durable knowledge. It is a library layer, not a CLI feature, and nothing else in CAVE depends on it.

## 27.1 Scenario inputs

`@cavelang/scenario` binds explicit CAVE-Q inputs against a frozen transaction-time and valid-time snapshot. Its `asOf` boundary follows CAVE-Q: UTC years, months, and days include their whole period, and a UUID includes that exact append. Invalid dates are rejected. A definition names the queries, the variable each one selects, the expected kind and unit, the cardinality, and a policy for every awkward case: what to do when a value is missing, contested, retracted, or unresolved. Nothing silently chooses the first match. Unknown policy names are rejected before database access. A definition may also overlay hypothetical CAVE claims, applied inside a savepoint that is rolled back before any evaluator runs, so "what if the team were twelve people" binds an exact integer while the store still says eight. The returned record is plain immutable data that remembers the supporting belief rows. If the store changes while those inputs are being read, binding fails with `snapshot-changed`; retry before evaluating. Backdated imports also invalidate the attempt. The evaluator never receives a record assembled across different database states.

## 27.2 Solver-neutral models

`@cavelang/solver` is a dependency-free TypeScript model for Boolean, bounded-integer, exact-real, and finite-enum variables. It distinguishes hard constraints, explicitly weighted soft constraints, and lexicographically ordered objectives. Exact reals are decimal strings or rationals normalized with big integers; no decimal is routed through floating point. CAVE confidence never becomes an optimization weight implicitly: confidence, probability, cost, and preference remain different concepts. Division always returns an exact real, including for integer operands, so capability checks require rational support before calling the backend.

The workflow API gives feasibility, optimization, counterexample, and bounded sensitivity distinct semantics over one validated model, one snapshot context, one set of resource limits, and one result vocabulary. Results are disjoint: **satisfied** and **optimal** carry assignments, **unsatisfied** requires a proof of infeasibility, and a timeout or backend failure remains **unknown**. Assignments are deterministic, ordered by stable variable id with false before true, smaller numbers first, and enum values in lexical order; authored objectives come first, soft weights next, and generated tie-breaks last. Sensitivity checks an explicit typed sample list and reports adjacent transitions and contiguous unknown regions rather than interpolating through timeouts.

## 27.3 The Z3 adapter

`@cavelang/solver-z3` is the optional Node.js adapter to the official threaded Z3 WebAssembly package. It loads lazily, queues checks through one process runtime, tracks named hard constraints for unsatisfiable cores, preserves exact rational arithmetic, applies bounded execution, and requires explicit worker shutdown. Its separate `cave-solver-workflow` binary is an allowlisted fixture that accepts bounded typed flags for one architecture example and no raw model or SMT-LIB input:

```
$ cave-solver-workflow architecture optimization --team-size 10 --deployment-frequency 6
$ cave-solver-workflow architecture sensitivity --team-size 10 --from 1 --to 12
```

The normal CAVE CLI, the MCP server, the browser playground, and the knowledge kernel do not depend on Z3.

## 27.4 Explanations and recording

Solver explanations map assignments, evaluated constraints, objective contributions, and unsatisfiable cores back to model locations, scenario inputs, and exact CAVE evidence rows. Counterexample reports also state the assumptions, bounded domains, and theories in scope. Theories account for expressions and literals as well as declared variable sorts. The model digest, solver version, resource limits, and frozen snapshot travel with the report.

Solve calls copy and freeze the model before backend execution. Explanation calls also copy their context and reject mismatched replay digests before solving, preserving submitted identity and evidence across later caller edits. Workflows capture options and context for the whole operation; sensitivity also captures its request, keeping later samples independent of caller edits. Unknown option or limit names, non-Boolean core flags, and invalid sensitivity modes fail before solving. Configured size limits also apply during hashing and replay checks; they do not change model identity. Linear-subset recognition excludes division by computed zero constants, using exact rational arithmetic rather than floating-point approximations.

Rendering an explanation is read-only. Constraint evaluation uses an explicit stack and preserves short-circuiting, so deep valid expressions do not become indeterminate from call-stack exhaustion. The `maxExplanationBits` limit conservatively bounds local fraction sizes before allocation. Exceeding it makes the affected constraint indeterminate while preserving the backend result; a larger budget permits retry. This per-operation guard is complemented by `maxExplanationWork`, which accumulates estimated-bit charges across local arithmetic in one report (default: 10,000,000). Exhaustion marks affected evaluations indeterminate; a fresh report resets the allowance. Neither guard is a wall-clock or process-memory limit, and preparation and formatting remain outside the arithmetic counter.

The workflow CLI flags are `--max-explanation-bits` and `--max-explanation-work`. Historical reports retain their recorded limits without adding newer defaults during replay. Scenario input records retain a digest of the complete binding definition. Explanation conversion checks it together with the record's content digest, so changed queries or policies cannot be attached to older evidence. Older records without that definition digest must be rebound before this conversion.

Solver output is not a write. An explicit, atomic, idempotent record transition can preserve an immutable run, but recommendations, human decisions, action audit records, and external-effect audit records use separate versioned identities. IDs must form exactly one namespaced entity; syntax that changes its spelling or introduces another claim is rejected without recording anything. Retracting a record makes it unavailable but does not free its ID: only identical content can be explicitly restored, and changed content requires a new ID. Artifact data must be ordinary JSON: sparse arrays, cycles, and objects such as dates or maps are rejected before writing, rather than silently losing their contents. Replay reports model or solver incompatibility instead of silently re-evaluating. Passing proposed parameters into an action still rechecks the current declaration, parameters, premises, shape gate, and transaction boundary before the governed write engine appends anything.

**What a solver proves**
A solver proves statements only inside the selected model and snapshot. **Optimal** means optimal under the declared inputs and objectives, not objectively best in the world. Encode one narrow decision with measurable inputs and test it against known cases before trusting the number.

> **In short.** Scenario bindings turn CAVE-Q answers into typed, replayable inputs with explicit policies for missing and contested values. The solver package is an exact, backend-neutral model with disjoint result states; Z3 is an opt-in adapter. Explanations trace back to evidence rows, and recording a run is an explicit step separate from any decision.

CHAPTER 28

# Running a Store for Real

The earlier chapters explain what each surface does. This one is advice: habits that keep a store trustworthy over months, the security boundaries to know about, what to expect from performance, and the command that checks an installation.

## 28.1 Habits

- **Keep canonical text under version control and treat the store as a working index.** The annotated export is a complete replica; a store can be rebuilt from it on any machine (Chapter 24).
- **Declare domain verbs and their inverses in a small prelude** that every file starts from, and keep extensions rare.
- **Use source contexts consistently**, so that resolution policy can tell actors apart and reports can cite them.
- **Prefer actions over free-form appends for consequential writes**, from people and agents alike.
- **Run `cave check` in continuous integration**, and `cave eval` whenever an extraction prompt, model, or instruction set changes.
- **Use `--resolve` only where a single winner is required**; keep default reads plural.
- **Use `--as-of` and `--at` explicitly in any report that depends on time**, so re-rendering it later gives the same answer.
- **Keep hooks and agent commands out of the store.** Claims may name them; they never contain executable configuration.
- **Take exact snapshots with `cave backup`**, record their hash, verify them independently, and periodically test `cave restore` into a fresh path.
- **Decide what not to record.** History is permanent; secrets and selectively erasable data stay out (Chapter 8).

## 28.2 Security boundaries

The boundaries are intentionally visible rather than implicit. The MCP server separates read, ephemeral evaluation, durable recording, and effect-capable action classes, and exact tool allowlists narrow further. Actions validate parameters and preconditions, and run inside the shape gate. Hook substitution is shell-quoted, but hook configuration is executable operator-controlled code and deserves review like any script. The read-only server should stay bound to localhost unless placed behind your own access layer; its sensitivity ceiling limits publication but is not authentication or encryption. Sensitivity labels are routing metadata, and a lower ceiling is a view, never a sanitizer.

## 28.3 Performance

Everything is designed for local scale: one store, one machine, indexes on every field a pattern can touch, and full-text search for the rest. Ordinary claim reads and appends are fast well past a hundred thousand rows. The costs that grow faster are transitive queries over deep graphs and large alias closures, which are recursive walks, and the health check over a store with many expectations. Query pages are bounded at a thousand matches so that an agent cannot ask for the whole store in one call. Measure on a representative store before reaching for distributed infrastructure; the repository keeps a deterministic performance benchmark with recorded budgets for exactly this reason.

## 28.4 Checking an installation

`cave doctor` is a read-only diagnostic for the runtime, the installed package layout, optional configuration, and a store's health. Its output never includes paths, hook contents, URLs, or environment values, so it is safe to paste into an issue:

```
$ cave add --db roastery.db roastery.cave
added 33 claim(s), 0 edge(s)

$ cave doctor --db roastery.db
cave doctor <token>
PASS Node <token> satisfies ^24.16.0 || ^26.1.0
PASS SQLite <token> supports FTS5, JSON functions, foreign keys, and extension loading
PASS Grammar WASM and highlight query are installed
PASS pnpm is not required for this installed CLI
PASS CAVE schema 2 is compatible (33 claim(s))
PASS SQLite integrity and foreign-key checks passed
PASS Stored payloads, provenance, flags, confidence and sigma levels are valid
PASS Search entries match the stored claims
PASS No optional hooks file was configured
result: ready
```

The CAVE, Node, and SQLite version tokens reflect the installed CLI and runtime and vary between releases and supported installations.

Warnings such as a database that does not exist yet exit 0; an unsupported runtime, a broken grammar asset, a malformed hooks file, or a corrupt store exit 1. `--hooks <file>` validates a hook configuration without running it, and `--json` emits a versioned report.

For a current SQLite store, the `store.search` check compares search entries with the stored claims. Missing, duplicate, orphaned, or stale entries fail this check even when SQLite's integrity check passes. Doctor reports the problem without repairing the index or printing claim contents. Text-store load failures likewise omit private paths and source details; use `cave parse` on the file locally for detailed syntax errors.

## 28.5 Signals, exit codes, and errors

Every command exits 0 on success and 1 on a user-level problem. A simple failure, such as a missing file or a malformed line, puts the problem on standard error in one line. Status 2 is reserved for the dispatcher's own usage errors: an unknown command or help topic, which also prints the usage overview, and a stray argument to a command that takes none (`doctor`, `demo`, `version`, `help`). Every other argument problem is an ordinary status-1 problem, including an unknown option, a missing pattern for `cave query`, and a stray argument to a delegated command such as `cave serve`, which prints its own usage text alongside the message. Commands that report violations or step failures, such as `cave check`, `cave eval --min`, `cave report`, and `cave automate --once`, use the exit code to carry that result so they compose with shell pipelines and CI, and their findings are the report itself rather than a one-line error: `cave check` prints its report on standard output, and a rejected `cave add --check` prints a header and one line per violation on standard error, as Chapter 13 shows. Interrupting a long-running command with `SIGINT` or `SIGTERM` closes servers, watchers, and stores before the process exits with the conventional signal code. `CAVE_DEBUG=1` adds the stack trace to unexpected errors. For commands that open a store, `--db` defaults to `$CAVE_DB`, then to `cave.db` in the current directory, except `cave restore`, which always requires an explicit destination; `$CAVE_HOOKS` supplies a default hook configuration.

> **In short.** Commit the annotated export, keep a prelude, stamp sources, prefer actions, gate with `cave check` and `cave eval`, resolve only on purpose, anchor reports in time, keep executable configuration out of the store, and back up with verified snapshots. `cave doctor` checks an installation without exposing anything private.

PART

# Appendices

The command reference, the compact syntax card, and a map from this book to the specification sections.

CHAPTER 29

# Command Reference

Every command answers `--help`, and `cave help <command>` prints the same text with examples. For the commands that open a store, `--db` defaults to `$CAVE_DB`, then `cave.db` in the current directory; `cave restore` is the exception and requires an explicit destination, and commands such as `parse`, `highlight`, `eval`, `doctor`, `demo`, and `version` take no store or only an optional one. The path names a store by content: a SQLite file, or a CAVE text file replayed into memory for the commands that only read (Chapter 9); appending commands refuse text files, and a read never creates a missing database. This table is the map; the chapters are the territory.

| Command | Purpose | Chapter |
|---|---|---|
| `cave parse` | Lint CAVE text; `--json` dumps the parsed document. | 2 |
| `cave highlight` | Print CAVE text with ANSI colours from the tree-sitter grammar. | 26 |
| `cave add` | Append authored claims; `--strict`, `--check` (shape gate), `--no-src`. | 2, 13 |
| `cave import` | Replay exported text without provenance stamping. | 7, 8 |
| `cave query` | Run a CAVE-Q pattern; `--all`, `--aliases`, `--as-of`, `--at`, `--resolve`, `--sources`, `--limit`, `--cursor`, `--json`, `--jsonl`. | 9 |
| `cave search` | FTS5 full-text search over claims and comments; `--raw`, `--limit`, `--json`. | 9 |
| `cave resolve` | List contested facts with ranked candidates; `--policy` shows the effective policy. | 11 |
| `cave derive` | Declare and fire rules; `--dry-run`, `--full`, `--list`, `--retract`. | 17 |
| `cave act` | Execute, `--declare`, `--list`, or `--retract` actions; `--hooks`, `--dry-run`, `--no-check`. | 18 |
| `cave automate` | The event loop; `--once`, `--declare`, `--list`, `--retract`, `--hooks`, `--agent`. | 19 |
| `cave check` | Knowledge health report; exit 1 on violations; `--stale`, `--json`. | 13 |
| `cave backup` | Exact verified store snapshot; `--verify`, `--sha256`. | 8 |
| `cave restore` | Verify and atomically restore a snapshot; `--force`. | 8 |
| `cave generate` | Versioned TypeScript client from `EXPECTS` claims. | 13 |
| `cave suggest-alias` | Propose same-entity candidates; `--min`, `--agent`, `--write`. | 12 |
| `cave sync` | Merge a store or annotated text by row identity; `--as`, `--into`, `--no-record`. | 24 |
| `cave export` | Canonical text; `--current`, `--tx`, `--max-sensitivity`, `--out`. | 7, 8, 24 |
| `cave serve` | Read-only local browser page; `--port`, `--host`, `--max-sensitivity`. | 22 |
| `cave report` | Render a cited Markdown template; `--resolve`, `--as-of`, `--at`, `--aliases`. | 21 |
| `cave mcp` | Serve the store to MCP clients; `--read-only`, `--permissions`, `--tools`, `--hooks`, `--src`. | 23 |
| `cave ingest` | Model-driven ingestion of files and URLs; `--agent`, `--stdout`, `--instructions`, `--lenient`. | 15 |

| | | |
|---|---|---|
| `cave eval` | Golden-fixture extraction and reconstruction evals; `--agent`, `--runs`, `--judge`, `--min`. | 16 |
| `cave connect` | Deterministic structured ingestion; `--map`, `--key`, `--prune`, `--watch`, `--query`; no source runs the declared ones, `--list` shows them. | 14 |
| `cave reconstruct` | Best-first reconstruction from seed cues; `--trace`, `--steps`, `--agent`. | 20 |
| `cave doctor` | Runtime, installation, configuration, and store diagnostics. | 28 |
| `cave demo` | Narrate the reconstruction demo on an in-memory store. | 20 |
| `cave version` | Print the version. | |
| `cave help` | The overview, or one command's options and examples. | |

Two environment variables matter: `CAVE_DB` sets the default store and `CAVE_HOOKS` the default hook configuration. `CAVE_DEBUG=1` keeps stack traces on unexpected errors and restores Node runtime warnings, which the binary suppresses by default. `NO_COLOR` disables the colours `cave export` prints on a terminal.

The optional `cave-solver-workflow` binary from the Z3 adapter package runs the allowlisted architecture fixture (Chapter 27).

For shell pipelines, `cave query --jsonl` prints one JSON bindings object per match, with variable names as keys and no comments or claim metadata. Pipe it to `jq -r '.repo'` to extract a variable named `?repo`. No matches produce no output; a fully bound match prints `{}`. The normal page limit applies, with `next: <token>` on stderr; pass the token to `--cursor` for the next page. Cursor continuation requires a persistent SQLite store; reopening a text store invalidates its snapshot. `--sources` returns the whole overlay, including for text stores. Choose either `--json` (the versioned page envelope) or `--jsonl`.

CHAPTER 30

# Syntax Card

The whole language on one page. Brackets mark optional parts.

```
subject VERB [NOT] object       [@context...] [#tag[:value]...] [@ N%] [!] [; comment]
subject HAS attribute: value [+/- delta [(Nσ)]] [@context...] [#tag...] [@ N%] [!] [;
comment]
metric IS value [+/- delta]     [@context...] ...

subject HAS                               ; incomplete prefix header, not a claim
  attribute: value                        ; -> subject HAS attribute: value
  other: value                            ; -> subject HAS other: value

parent VERB object
  VERB object2                            ; continuation: inherits the parent subject
  INVERSE-VERB other                      ; continuation: parent lands in object position
  WHEN condition                          ; qualifier edge on the parent claim
  BECAUSE evidence                        ; qualifier edge on the parent claim

NEW-VERB IS verb ; X does what to Y       ; declare a verb
VERB REVERSE INVERSE-VERB                 ; one fact, two names; the left side is primary
OLD-VERB RENAMED-TO NEW-VERB              ; prefer NEW; storage identity stays OLD
type EXPECTS attribute [#unit:u]          ; shape as claims
type EXPECTS VERB [#cardinality:one]
a ALIAS b                                 ; same entity, two names

?x VERB ?y                                ; query pattern; _ is a wildcard
?x VERB+ ?y                               ; one or more hops
?x VERB* ?y                               ; zero or more hops
?x (A|B) ?y, ?x (A|B)+ ?y, ?x (A|B)* ?y   ; verb alternation paths
WHERE conf >= 0.8 | value < 10 kg | tx <= 2026-08-01
?a A ?b, ?b B ?c, ?c < 18                 ; joined premises (one query)
OPTIONAL ?c C ?d | SELECT DISTINCT ?a | ORDER BY ?c DESC

premise, premise, ?v op value => conclusion ; rule (cave derive)
action/name HAS action: `?param, premise => effect, effect`
automation/name HAS automation: `trigger => action/x, hook/y, "prompt"`
source/name HAS precedence: 3 | HAS reliability: 80%

@production            context (no space after @)
@src:file.md#L10-L12   source with a line span
@2026-Q3, @2026..2027  time point, time range (valid time)
@ 80%                  confidence (space after @); @ 0% retracts
20B -> 40B USD/yr      trajectory across a single closed range
~20B USD/yr            approximate value
#topic:auth            scoped tag; #security is a flat tag
#sensitivity:public    audience label (public < internal < confidential < restricted)
!                      importance
; comment              persisted with the claim
```

Disambiguation: `@` followed by a space is confidence, `@` followed by a name is a context; `#` always begins a tag and the first `:` inside it splits key from value; `:` in payload position binds attribute to value; `/` after a number means “per”, elsewhere it is entity scope. Backticks hold exact code-like text and double quotes hold natural language; inside either, nothing is special.

The central mental model is short: write one claim per line, append rather than overwrite, ask with the same shape using variables, and keep provenance and uncertainty explicit. Every larger part of CAVE is built from that.

CHAPTER 31

# Where the Rules Live

The normative specification is split across four skill files in the repository's `.claude/skills/` directory, and section numbers are preserved there. This book explains; the sections decide. Sections are normative unless marked legacy, draft, or non-normative.

| Skill | Sections | Covers |
|---|---|---|
| `cave-writing` | §3–§8, §11, §16, §22 | Syntax, lexical rules, verbs, `REVERSE` and `RENAMED-TO`, metadata, values and units, indentation, tags and topics, the grammar, the spec card. |
| `cave-extraction` | §14–§15, §21, §23 | Extraction rules, granularity, operating modes, the worked example, deterministic structured ingestion. |
| `cave-storage-query` | §9, §12–§13, §20, §24–§32 | Belief evolution, claim keys, provenance, sensitivity, source spans, CAVE-Q, storage, shapes, rules, actions, resolution, alias discovery, sync, automations, the read surface, reports, valid time. |
| `cave-design` | §0–§2, §10, §17–§19 | Status conventions, design goals, the claim model, the probabilistic layer, the historical draft grammar, the agent layer, rationale. |

## 31.1 Chapter to section

| Chapter | Sections |
|---|---|
| 1 A First Claim | §1, §2 |
| 2 Claims | §3, §4 |
| 3 Verbs | §5 |
| 4 Context, Tags, and Confidence | §6, §11 |
| 5 Values, Units, and Uncertainty | §7, §10 |
| 6 Indentation | §8 |
| 7 Belief That Only Grows | §9.1–§9.4, §12.3 |
| 8 Provenance | §9.5–§9.8, §13.2.2 |
| 9 Asking Questions | §12, §13.5 |
| 10 Time in the World | §32 |
| 11 When Sources Disagree | §26 |
| 12 One Entity, Many Names | §13.6, §27 |
| 13 Shape and Health | §20 |
| 14 Structured Data Without a Model | §23 |
| 15 Letting a Model Read | §14, §15 |
| 16 Measuring an Extraction | §18 (evals), the `eval` package |
| 17 Rules | §24 |
| 18 Actions | §25 |
| 19 Automations | §29 |

| | |
|---|---|
| 20 Reconstruction | §18 |
| 21 Documents That Cite Their Claims | §31 |
| 22 Looking at the Store | §30 |
| 23 Agents Over MCP | §9.5, §25.5, the `mcp` package |
| 24 Two Stores Become One | §28 |
| 25 How Claims Are Stored | §13 |
| 26 How the Pieces Fit | §19, `ARCHITECTURE.md` |
| 27 Formal Reasoning | the `scenario`, `solver`, and `solver-z3` packages |
| 28 Running a Store for Real | §9.6, §9.7, §25.4 |

The root `README.md` is a step-by-step tutorial on two other examples, a monorepo and a market watchlist, and `examples/family-history/README.md` pushes one set of notes through every surface. Package READMEs under `packages/` are the reference for each command and library.